MemGate: An Adaptive Memory Layer for Long-Running LLM Agents

Project Report submitted in partial fulfilment
of

Bachelor of Technology (B.Tech) in

Artificial Intelligence and Data Science (AI&DS)

by

Yash Ramchandani (SAP ID: 70562300097)
Simar Singh Khanuja (SAP ID: 70562200088)

Under the supervision of

Dr. Praveen Modi
(Assistant Professor, STME, Indore Campus)

SVKM's NMIMS University
(Deemed-to-be University)

SVKM'S
NMIMS
Deemed to be UNIVERSITY

SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING (STME)

2026–27

CERTIFICATE

SVKM'S
NMIMS
Deemed to be UNIVERSITY

SVKM's NMIMS University
(Deemed-to-be University)

This is to certify that the project entitled **"MemGate: An Adaptive Memory Layer for Long-Running LLM Agents"** has been done by **Yash Ramchandani** (SAP: 70562300097) and **Simar Singh Khanuja** (SAP: 70562200088) under my guidance and supervision & has been submitted in partial fulfilment of the degree of **B.Tech** in **Artificial Intelligence and Data Science** of STME, SVKM's NMIMS (Deemed-to-be University), Indore Campus, India.

_____

Dr. Praveen Modi
(Internal Guide)

_____

Examiner
(External)

_____

Dr. Vikas Khare
(Associate Dean, STME)

Date: _____ **Place:** Indore

Acknowledgement

We would like to express our sincere gratitude to all those who contributed to the successful completion of this project.

First and foremost, we extend our heartfelt thanks to our faculty mentor, **Dr. Praveen Modi**, Assistant Professor, for invaluable guidance, continuous support and insightful feedback throughout the course of this project. The mentor's review of our research paper, and the steady push for every claim to be backed by a measurement, shaped this work.

We are deeply grateful to SVKM's NMIMS University and the School of Technology Management & Engineering (STME) for providing the necessary resources, infrastructure and academic environment that made this work possible.

We would like to express our profound gratitude to **Dr. Anshuman Jaswal**, Director, NMIMS University Indore Campus. We also extend our sincere thanks to **Dr. Vikas Khare**, Associate Dean, School of Technology Management & Engineering (STME), Indore Campus, for his continuous support and motivation.

We would also like to thank the faculty members of STME for their teaching and support, which laid the foundation for this project, and the members of the review panels, whose questions at Review 1 led us to move the project's contribution from the memory architecture to the memory decision policy.

We thank the authors of the LoCoMo, REALTALK, BEAM, MemBench, PersonaChat and LongMemEval datasets for releasing them openly; this project collected no data of its own.

Finally, we are immensely thankful to our families for their unwavering support, patience and encouragement throughout our academic journey.

Yash Ramchandani (SAP: 70562300097)
Simar Singh Khanuja (SAP: 70562200088)

Abstract

Agents built on large language models (LLMs) now hold conversations that run for weeks and grow far longer than any context window. On every turn something has to decide which parts of that history go into the prompt. Re-sending the whole history makes each reply slower and costlier, a sliding window forgets the oldest messages whatever they contain, and uniform summarisation loses the names, numbers and dates that are asked about later. Existing memory systems combine tiers, summaries and forgetting, but they are evaluated end to end, so the value of each individual decision is unknown.

This project builds **MemGate**, a memory layer that sits between an agent and a frozen LLM and decides, for every message, whether to *keep* it, *compress* it, *forget* it, or *recall* it later, without training the base model. It is built as a measurement harness: a three-tier store runs under a fixed storage budget, and policies that differ in exactly one factor are compared on the LoCoMo benchmark with 95% confidence intervals resampled by conversation.

The main findings are as follows. (i) The usual accounting bounds the context but not the store; once the store is bounded, choosing *which* turns to forget gains $+9.37$ answer-recall points over oldest-first eviction at identical storage, fidelity and retrieval, while compressing evicted turns instead of dropping them loses 7.13 points. (ii) No rate–distortion crossover appears up to $182\times$ compression. (iii) Counting the embedding index in bytes reverses the ranking below about 40 KiB. (iv) A small LLM judge of salience loses to a hand-written heuristic, whereas a logistic-regression scorer over shallow features and a sentence embedding, trained on generated and human conversations, beats the heuristic by $+14.55$ points on held-out conversations and outperforms rebuilt MemGPT-style and Mem0-style memory at every budget tested. (v) On held-out chats of up to ten million tokens it beats retrieval on every corpus, and when million-token chats are typed into the application it keeps the answer for 51.9% of questions against 19.2% for retrieval.

The learned scorer ships in a working chat application that shows, for every message, the tier it went to and why. The work has been written up as a research paper with the project mentor as corresponding author.

Keywords: large language models, agent memory, context management, retrieval-augmented generation, learned salience scoring, rate–distortion, empirical evaluation.

Contents

Acknowledgement i

Abstract ii

List of Figures vi

List of Tables vii

Abbreviations ix

1 Introduction 1

1.1 Background of the project topic 1

1.2 Motivation and scope of the report 2

1.2.1 Motivation 2

1.2.2 Scope 2

1.3 Problem statement 3

1.4 Salient contribution 3

1.5 Organisation of report 4

2 Literature Survey 6

2.1 Introduction to the overall topic 6

2.1.1 LLM agents and their memory 6

2.1.2 Why longer context windows are not enough 6

2.1.3 Retrieval as the default baseline 6

2.1.4 Ideas borrowed from systems and psychology 7

2.1.5 How memory is evaluated 7

2.2 Survey of the most relevant work 7

2.2.1 Memory systems for agents 7

2.2.2 Compressing the prompt or the cache 8

2.2.3 Benchmarks and theory 9
2.3 Comparison of the surveyed work 9
2.4 Research gap and problem definition 10

3 Methodology and Implementation 12
3.1 Block diagram 12
3.2 Hardware description 12
3.3 Software description 14
3.3.1 Technology stack 14
3.3.2 Module structure 14
3.3.3 Tiers as a lifecycle 15
3.3.4 Policies 15
3.3.5 Scoring a message 16
3.3.6 The storage budget and eviction 18
3.3.7 Context assembly and store maintenance 19
3.3.8 Algorithms 20
3.4 Experimental design 20
3.4.1 One variable per comparison 20
3.4.2 Evaluation metrics 22
3.4.3 Datasets and set-up 22
3.4.4 Two defects found and fixed 22
3.5 The chat application 23
3.5.1 Features 23
3.5.2 Server 24
3.6 Testing and verification 24

4 Results and Analysis 27
4.1 Storage was never budgeted 27
4.2 Selection pays; compression does not 27
4.3 Statistical significance 28
4.4 The rate–distortion crossover is not reached 28
4.5 The embedding index is not free 29
4.6 Scorer analysis 31

4.7 A learned scorer across two corpora 32
4.8 A negative result on LongMemEval 33
4.9 End-task accuracy 34
4.10 Against MemGPT-style and Mem0-style memory 35
4.11 Chats of up to ten million tokens 36
4.12 Ten times the training data 36
4.13 Stress and scale through the application 37
4.14 Standards followed 38

5 Advantages, Limitations and Applications 40
5.1 Advantages 40
5.2 Limitations 41
5.3 Applications 41

6 Conclusion and Future Scope 43
6.1 Conclusion 43
6.2 Future scope 44

References 45

A Soft Code Flowcharts 50

B Data Sheets 52

C List of Components 53

D Plagiarism and AI-Content Check Report 54

List of Figures

3.1 Block diagram of MemGate. 13
3.2 How the shipped scorer is built. 18
3.3 How the application scores a message (GuardedScorer). 18
3.4 Where the keep, compress and forget decisions are taken. 19
3.5 The application tags each message with its tier and score; the breakdown shows why the learned scorer kept a dated fact. 24
3.6 The memory panel: what each tier holds, with the score breakdown of a stored note. 25
3.7 The Compare tab: the same question answered from MemGate, a sliding window and RAG at the same budgets. 25
3.8 A Word file uploaded into a chat. 26

4.1 Recall against the storage budget for RAG, P1-A (compress) and P1-S (drop). 28
4.2 Answer recall as the compression ratio grows to 182×. 30
4.3 Answer recall with the storage budget counted in bytes, the index included. 31
4.4 Answer recall by scorer: recency, heuristic, LLM judge and learned. . . 32
4.5 Reader accuracy by memory policy, with the closed-book floor and the oracle ceiling. 35
4.6 Answer recall against the store budget for MemGate and the rebuilt MemGPT-style and Mem0-style memories. 36

A.1 Flowchart of the write path in the application: how one message is cleaned, gated, stored and kept within budget. 50
A.2 Flowchart of the read path in the application: how a reply is assembled from memory. 51

List of Tables

2.1 Comparison of the main related systems and the gap each leaves. 10

3.1 Computing resources used and required. 14
3.2 Technology stack. 14
3.3 Modules of the MemGate library (`memgate/memgate/`). 15
3.4 The policies compared. 16
3.5 Controlled contrasts: each changes exactly one factor. 21

4.1 Recall against the storage budget S in tokens, context budget 2048. . . . 28
4.2 Paired bootstrap (10,000 resamples) by conversation, with exact McNemar, at $S = 4096$. 29
4.3 Answer recall (%) against compression ratio at $S = 2048$. 29
4.4 Answer recall (%) with the budget in bytes, charging 1,536 B per retrievable item for its embedding. 30
4.5 Component ablation at $S = 4096$: change from the full system. 31
4.6 Single-component scorer swap at $S = 4096$. 32
4.7 Learned scorer on held-out conversations of two corpora, $S = 4096$. . . 33
4.8 The pooled learned scorer (P3) against the P1 heuristic across the store budget S, answer recall (%). 33
4.9 End-task accuracy through one fixed reader at $S = 4096$ over all 715 answer-recoverable questions. 34
4.10 Answer recall (%) on LoCoMo against rebuilt MemGPT-style and Mem0-style memory. 36
4.11 Held-out answer recall (%) at each corpus's store budget, 5-fold by conversation. 37
4.12 Retraining: change against the earlier recipe in three reshuffled fold orders, held out. 37
4.13 Whole BEAM chats typed into the application one message at a time, $S = 26{,}000$; answer recall (%). 38

4.14 Answer recall (%) by the position of a question's evidence, $S = 4096$. . 38

B.1 Datasets used. 52
B.2 Models and tokeniser used. 52

C.1 Software components: main files of the repository. 53
C.2 Software libraries. 53

D.1 Turnitin check summary 54

Abbreviations

Abbreviation	Full form
API	Application Programming Interface
AUC	Area Under the (ROC) Curve
BEAM	Benchmark for long-context memory (100K to 10M-token chats)
CI	Confidence Interval
CPU / GPU	Central / Graphics Processing Unit
F1	Harmonic mean of precision and recall
FIFO	First In, First Out
JSON	JavaScript Object Notation
KiB	Kibibyte (1,024 bytes)
KV cache	Key–Value cache of a transformer's attention
LLM	Large Language Model
LoCoMo	Long Conversational Memory benchmark
LOCO	Leave-One-Conversation-Out (cross-validation)
LR	Logistic Regression
MiniLM	all-MiniLM-L6-v2 sentence embedding model
MLP	Multi-Layer Perceptron
NER	Named-Entity Recognition
n.s.	not significant
ONNX	Open Neural Network Exchange (model format)
P0	Sliding-window policy (recency only)
P1	MemGate with the hand-written heuristic scorer
P1-A	MemGate, adaptive mode: compress on eviction
P1-S	MemGate, select mode: drop on eviction
P2	MemGate with an LLM judge as scorer
P3	MemGate with the learned (logistic) scorer
PDF	Portable Document Format
QA	Question Answering
RAG	Retrieval-Augmented Generation (store-all, FIFO baseline)
REST	Representational State Transfer (web API style)
S	Storage budget of the memory store
B	Context budget of the assembled prompt
UI	User Interface
UTF-8	Unicode Transformation Format, 8-bit

Chapter 1

Introduction

This chapter introduces the problem that MemGate addresses: how a long-running agent built on a large language model should decide what to remember. It explains why larger context windows do not remove the need for that decision, sets out the motivation and scope of the project, states the problem precisely, lists the contributions, and describes how the rest of the report is organised.

1.1 Background of the project topic

Large language models (LLMs) are now used as the core of *agents*: programs that plan, call tools, reflect on their own mistakes and carry on working over many sessions [1–4]. Coding assistants, research assistants, tutoring systems and personal assistants are all expected to remember what was said days or weeks earlier. An LLM, however, has no memory of its own between calls. Everything it knows about the conversation has to be placed in its prompt, inside a fixed *context window* [5].

The simplest way to give an agent memory is to append every message to a growing prompt and send the whole history back to the model on every turn. This works at first but does not scale. The cost of self-attention grows with the square of the sequence length, and the key–value (KV) cache that the model keeps grows with every token, so each reply becomes slower and more expensive until the history no longer fits at all. Techniques that stretch the window, such as position interpolation [6], delay that point but do not remove it.

A larger window does not solve the quality problem either. Across frontier models, accuracy drops by 30–50% well before the advertised limit [7]; synthetic tests find effective context lengths far shorter than the claimed ones [8]; and how much a model uses a passage depends on where it sits in a long input [9]. These effects come from attention over long sequences rather than from missing training data. What limits an agent is therefore *curation*, the choice of what goes into the prompt, rather than raw capacity.

Several systems already manage agent memory. MemGPT [10] pages a virtual context in and out of external storage, MemoryBank [11] applies a forgetting curve, Generative Agents [12] rank memories by recency, importance and relevance, and Mem0 [13] extracts and updates facts with an LLM. Retrieval-augmented generation (RAG) [14]

stores everything and retrieves the passages most similar to the query. Agent architectures now treat memory as a module in its own right [15]. These designs show that tiered memory with forgetting works in practice. They are, however, evaluated as complete systems, so it is not known how much each individual decision inside them (what to keep, what to compress, what to forget) is actually worth.

1.2 Motivation and scope of the report

1.2.1 Motivation

Poor curation has a measurable price. On the LongMemEval benchmark, a model given oracle retrieval of the right evidence reaches about 92% accuracy, while the same model choosing its own evidence reaches about 58% [16]. Given the right evidence the model answers correctly, so the 34-point gap is a failure of *selection*, not of reasoning. A recent survey of agent memory puts it plainly: current systems "do not solve the fundamental challenge: deciding what to remember and what to forget" [17]. A rate–distortion view of these decisions [18,19] has been proposed [20], but as an analytical framework rather than a measured system.

At Review 1 the panel asked whether a three-tier memory layer was still novel, since commercial systems already ship one. The honest answer was that it is not. The project therefore moved its contribution one level deeper, to the *decision policy* itself: building a layer in which each decision can be isolated, swapped and measured, and then using it to find out which parts of the familiar design actually earn their cost.

1.2.2 Scope

In scope.

- A memory controller that scores, compresses, stores, forgets and retrieves conversational memory in front of a frozen LLM, which is treated as a black box and never trained or fine-tuned.
- A three-tier store (recent verbatim turns, compressed gists, durable facts with embeddings) held under a fixed storage budget.
- Several interchangeable scoring policies: recency only, a hand-written heuristic, an LLM judge, and a learned scorer.
- Evaluation on public long-conversation benchmarks (LoCoMo, REALTALK, BEAM, MemBench, PersonaChat, LongMemEval) with statistical tests that respect the clustering of questions by conversation.
- A working chat application built on the same code, so that the memory decisions

can be seen and tested by a user.

Out of scope. Training or fine-tuning the base LLM; image and audio memory (the project is text only); production multi-user serving with accounts and billing.

1.3 Problem statement

> *Given a conversation that grows without bound, a frozen LLM that can only see a bounded context of B tokens, and a memory store that may hold at most S tokens (or bytes), design and evaluate a memory decision policy that chooses, for every message, whether to keep it verbatim, compress it, forget it or recall it, so that the answers to later questions reach the model as often as possible. Measure the contribution of each decision separately, against controls that differ from it in exactly one factor.*

The problem has two parts. The *engineering* part is to build the layer so that it is fast, never exceeds its budget, and works on real input. The *research* part is to find out, with proper statistics, whether scored forgetting beats oldest-first forgetting, whether compressing beats dropping, whether a learned scorer beats a hand-written one, and whether the answer survives a change of corpus.

1.4 Salient contribution

The project makes the following contributions.

1. **A storage-budgeted evaluation protocol.** Bounding only the assembled context charges a store-everything baseline nothing for what it keeps. MemGate bounds the whole store, which changes the ranking of the policies (Section 4.1).
2. **Selection pays; compression does not.** At identical storage, fidelity and retrieval, scored forgetting gains $+9.37$ answer-recall points over FIFO, while compressing an evicted turn instead of dropping it costs 7.13 points (Sections 4.2 and 4.3).
3. **A test of the rate–distortion crossover** up to $182\times$ compression, which is not reached (Section 4.4).
4. **Byte accounting.** The embedding index outweighs the text it indexes by about $12\times$ and reverses the policy ranking below roughly 40 KiB (Section 4.5).
5. **A negative result for LLM-judged salience.** A 0.5B judge loses to a hand-written heuristic by 5.87 points (Section 4.6).

6. **A learned scorer** trained on generated and human conversations, with the direction in which it transfers and the one in which it does not (Section 4.7).
7. **End-task accuracy** through a fixed reader, showing how much of the recall advantage becomes correct answers (Section 4.9).
8. **Comparison with MemGPT-style and Mem0-style memory** rebuilt in the same harness (Section 4.10).
9. **Scale.** A test on chats of up to ten million tokens and a scorer retrained on 14.6 times the data (Sections 4.11 and 4.12).
10. **A working application.** A chat app that runs the shipped scorer, shows every keep, compress and forget decision with its reasons, accepts pasted text and uploaded documents, and was stress-tested on million-token chats and 200 simultaneous users (Sections 3.5 and 4.13).
11. **A research paper**, "Selection Beats Compression: Memory Decision Policies for LLM Agents Under a Bounded Store", prepared in conference and journal versions.

Several of these results are negative or corrective. We regard that as the useful part: described only as a three-tier memory layer, MemGate would repeat work others have done, whereas measured against controls it shows which parts of that design are worth their cost.

1.5 Organisation of report

The rest of the report is organised as follows. **Chapter 2** surveys the literature on agent memory, prompt and cache compression, retrieval and evaluation, compares the main systems, and identifies the research gap. **Chapter 3** describes the methodology and implementation: the block diagram, the computing resources used, the scoring, routing and budget algorithms with their equations and flowcharts, the training of the learned scorer, the experimental design, and the chat application. **Chapter 4** presents the results and their analysis, with confidence intervals, together with the standards followed. **Chapter 5** discusses advantages, limitations and applications, and **Chapter 6** concludes and sets out the future scope. The appendices contain the flowcharts of the software, data sheets of the datasets and models, the list of software components, and the papers prepared from this work.

Summary

LLM agents need a policy that decides what to remember, because larger context windows move the problem rather than remove it. Existing memory systems are evaluated whole, so the worth of each decision is unknown. MemGate is built to measure those decisions one at a time under a bounded store, and the rest of this report describes how it was built and what it found.

Chapter 2

Literature Survey

This chapter reviews the work on which MemGate builds. It first gives the background of the overall topic: why LLM agents need memory, how long contexts fail, and the older ideas from computer systems and psychology that memory layers borrow. It then surveys the most relevant papers one by one, compares them in a table, and ends by identifying the research gap that defines the problem addressed in this project.

2.1 Introduction to the overall topic

2.1.1 LLM agents and their memory

Large language models learn to perform new tasks from instructions and a few examples placed in the prompt [5]. Building on this, agent frameworks interleave reasoning with actions such as tool calls [1] and let a model learn from its own failures by writing verbal reflections into its context [2]. Two broad surveys describe how such agents are assembled from a profile, a planner, a memory and an action module [3, 4], and the CoALA framework treats memory explicitly as working, episodic, semantic and procedural stores around the model [15]. A survey devoted to agent memory organises the field by memory source, form and operation, and concludes that deciding what to remember and what to forget remains the open problem [17].

2.1.2 Why longer context windows are not enough

One response is to make the window larger. YaRN extends the context of existing models by interpolating rotary position embeddings [6]. Larger windows, however, are used unevenly. Models make best use of information at the beginning and the end of a long input and lose it in the middle [9]; the RULER benchmark finds effective context lengths far below the advertised ones [8]; and a 2025 study of 18 frontier models reports accuracy losses of 30–50% well before the documented limit [7]. Long-context benchmarks such as LongBench measure this capacity directly, without a memory layer [21].

2.1.3 Retrieval as the default baseline

Retrieval-augmented generation (RAG) keeps external text and puts the passages most similar to the query into the prompt [14]. Dense passage retrieval made this practical

with learned sentence embeddings [22], and RETRO showed that retrieval from a very large store can stand in for model size [23]. Sentence-BERT [24] and the distilled MiniLM model [25] provide the small, fast embeddings that this project uses. RAG stores everything and never forgets, so it is the natural control against which a forgetting policy must be judged.

2.1.4 Ideas borrowed from systems and psychology

Memory layers for agents reuse older ideas. The multi-store model of human memory separates a short-term store from a long-term store with control processes between them [26]. Operating systems face the same choice when physical memory is full: Belady's study of page replacement [27] and Denning's working-set model [28] are the classical treatments, and MemGPT makes the analogy explicit [10]. Rate–distortion theory [18, 19] gives the language for trading the size of a stored representation against its fidelity, and has recently been proposed as a framework for agent memory [20].

2.1.5 How memory is evaluated

LoCoMo provides very long multi-session dialogues with question–answer pairs whose evidence turns are marked [29]. LongMemEval separates several abilities of chat assistants over long histories and measures the gap between oracle and self-selected evidence [16]. REALTALK applies LoCoMo's annotation scheme to 21 days of real messages between people [30]; BEAM scales conversations to ten million tokens [31]; and MemBench tests factual and reflective memory of agents [32]. LLM-as-a-judge evaluation [33] is widely used for open-ended answers, while extractive question answering is scored by string match [34]. For comparing two systems on the same questions, McNemar's test [35, 36] and the bootstrap [37] are standard; when questions are grouped into conversations, the resampling must respect that grouping [38]. Finally, studies of data leakage show how easily an evaluation can be contaminated by information that would not be available at prediction time [39, 40], a failure this project met and fixed (Section 3.4.4).

2.2 Survey of the most relevant work

This section discusses the most relevant papers in more detail, grouped by the decision they make.

2.2.1 Memory systems for agents

MemGPT (Packer et al., 2023) [10] treats the context window like the main memory of an operating system. The model itself decides, through function calls, what to page

out to an archive and what to bring back, and a recursive summary keeps a compressed record of evicted messages. MemGPT shows that an agent can work well beyond its window, and it was chosen as the base paper of this project. Its limitation, for our purpose, is that every memory decision is left to the LLM, so there is no explicit importance policy whose value can be measured on its own.

MemoryBank (Zhong et al., 2024) [11] adds a long-term memory with an Ebbinghaus-style forgetting curve: memories decay with elapsed time and are reinforced when recalled. It produces human-like forgetting, but it forgets by age rather than by importance and does not hold the store to a fixed budget.

Generative Agents (Park et al., 2023) [12] record a stream of observations for each simulated character and retrieve from it by a weighted sum of recency, importance (rated by the LLM) and relevance, with periodic reflections that summarise the stream. The design sustains believable long-horizon behaviour, but the stream is never bounded and the system is not built to reduce tokens or cost.

A-MEM (Xu et al., 2025) [41] organises memories as linked notes in the style of a Zettelkasten. Each new memory is written as a structured note, linked to related notes, and may cause older notes to be revised. On LoCoMo it improves multi-hop reasoning while using fewer tokens. Its focus is on how notes are linked rather than on what to forget under a strict budget.

Mem0 (Chhikara et al., 2025) [13] is a production-oriented memory layer. An LLM extracts candidate facts from each exchange and chooses among ADD, UPDATE, DELETE and NOOP against the nearest stored facts. It reports large savings in latency and tokens against full-context baselines on LoCoMo. Because the extraction and update decisions are made by an LLM end to end, the contribution of each decision cannot be separated, which is why Section 4.10 rebuilds its memory structure inside the MemGate harness.

2.2.2 Compressing the prompt or the cache

LLMLingua (Jiang et al., 2023) [42] compresses a given prompt by removing tokens that a small language model finds predictable, under a budget controller, reaching up to about 20$\times$ compression with small losses. It compresses what it is given but does not decide what to store or forget across sessions.

Gist tokens (Mu et al., 2023) [43] train a model to compress an instruction into a few virtual tokens that can be cached and reused. **AutoCompressors** (Chevalier et al., 2023) [44] recursively compress long documents into summary vectors used as soft prompts, and the **in-context autoencoder** (Ge et al., 2024) [45] packs a context into a small number of memory slots. All three require training the model and produce opaque

vectors rather than readable memories.

StreamingLLM (Xiao et al., 2024) [46] observes that models attend heavily to the first few tokens ("attention sinks") and keeps those plus a rolling window of recent tokens in the KV cache, giving stable generation over millions of tokens. **H_2O** (Zhang et al., 2023) [47] keeps the "heavy-hitter" tokens that receive the most attention and evicts the rest. Both operate inside one forward pass over a contiguous context; anything they evict is gone for good. Compressive Transformers [48], Memorizing Transformers [49] and LongMem [50] go further and change the model so that it can compress or index its own past activations.

2.2.3 Benchmarks and theory

LoCoMo (Maharana et al., 2024) [29] contains ten very long conversations, generated by LLM agents with personas and event graphs and then checked by people, with about 1,500 questions in five categories whose evidence turns are marked. The marked evidence is what lets a retrieval-level metric be computed without running a generative model, and LoCoMo is the main corpus of this project.

LongMemEval (Wu et al., 2024) [16] tests five memory abilities (information extraction, multi-session reasoning, temporal reasoning, knowledge updates and abstention) over histories of up to about 1.5 million tokens. Its key observation for this project is the 34-point gap between oracle and self-selected evidence.

REALTALK (Lee et al., 2025) [30] collects 21 days of messaging between real people and annotates it in LoCoMo's scheme. Because LoCoMo is generated by an LLM, REALTALK is the natural test of whether a scorer learned on LoCoMo has learned what people ask about or only what generated dialogue looks like.

The agent memory survey (Zhang et al., 2024) [17] organises the field and states the open problem. **The rate–distortion framework** (Colaco and Lahjouji, 2026) [20] argues that keeping, compressing and forgetting are one decision about how much to retain and at what fidelity under a budget, and predicts that compression should win once the budget is tight enough. It is a framework rather than a measured system, and Section 4.4 tests its prediction.

2.3 Comparison of the surveyed work

Table 2.1 compares the main systems along the lines used in the Review 1 presentation, with the gap each one leaves and how MemGate addresses it.

Table 2.1. Comparison of the main related systems and the gap each leaves.

Work	Data / benchmark	Technique	Limitation	Addressed in MemGate
MemGPT [10] (2023)	Multi-Session Chat; document QA	Tiered memory; LLM pages context via function calls	Memory decisions left to the LLM; no measurable importance policy	An explicit, swappable scoring policy, measured on its own
MemoryBank [11] (2024)	Long-term dialogue (SiliconFriend)	Vector memory with Ebbinghaus forgetting curve	Forgets by elapsed time, not importance; no budget	Forgets by scored utility under a hard storage budget
Generative Agents [12] (2023)	Sandbox simulation	Memory stream; recency, importance and relevance; reflection	Unbounded stream; not built to cut tokens	Bounds the store; keeps retrieval by relevance
A-MEM [41] (2025)	LoCoMo	Linked notes, dynamic revision	Focus on linking, not on budgeted forgetting	Budget-driven forgetting, compared on LoCoMo
Mem0 [13] (2025)	LoCoMo	LLM fact extraction with ADD/UPDATE/DELETE	Decisions made end to end by an LLM	Rebuilt in the same harness and compared at equal budgets
StreamingLLM [46] (2024)	Long-stream language modelling	Attention sinks plus rolling KV cache	Evicted content is gone for good	Evicted items can live on as gists or facts and be recalled
H_2O [47] (2023)	OPT and LLaMA tasks	Keeps heavy-hitter tokens in the KV cache	Works inside one forward pass	Works across sessions, at the conversation level
LLMLingua [42] (2023)	GSM8K, BBH and others	Token-level prompt compression	Compresses, but does not decide what to keep	Tests compression against dropping inside one policy
Gist tokens [43] (2023)	Instruction following	Learned virtual tokens	Needs training; fixed instructions only	No training of the base model; readable memory
AutoCompressors [44] (2023)	Long-context language modelling	Recursive summary vectors	Opaque vectors; no explicit forgetting	Readable gists plus a retrievable fact store
Memory survey [17] (2024)	Survey	Taxonomy of agent memory	No system	Measures several of its open questions
Rate–distortion [20] (2026)	Theory	Memory as rate–distortion	Framework, not a measured system	Tests its crossover prediction to 182×

2.4 Research gap and problem definition

The survey leads to four observations.

1. **Tiered memory with forgetting is no longer new.** MemGPT, Mem0, A-MEM and others already ship it. A project that only builds another three-tier layer would repeat their work.
2. **No one has measured the decisions separately.** Each system is evaluated as a whole, with its own model, prompts and budgets, so a reader cannot tell whether its gains come from what it keeps, how it compresses, how it retrieves, or the model it uses.
3. **The accounting is incomplete.** Evaluations cap the context sent to the model but leave the store unbounded, which lets a store-everything baseline keep its whole history for free. Byte costs, including the embedding index, are not counted.
4. **Salience scoring is ad hoc.** Importance is rated by hand-written rules or by asking an LLM, and whether a learned scorer would do better, and whether it would carry over from generated to human conversation, has not been tested.

These observations define the problem stated in Section 1.3: build a memory layer whose keep, compress, forget and recall decisions can be isolated and swapped; hold it to a storage budget counted in tokens and in bytes; and measure each decision, including a learned scorer, against controls that differ from it in one factor only, with statistics that respect the grouping of questions into conversations.

Summary

The literature offers many working memory systems, several ways to compress prompts and caches, and good long-conversation benchmarks. What it lacks is a controlled measurement of the individual memory decisions under a bounded store, a byte-level accounting of their cost, and a tested learned scorer. MemGate is designed to fill exactly that gap.

Chapter 3

Methodology and Implementation

This chapter describes how MemGate was designed and built. It starts with the block diagram of the system, describes the computing resources used (MemGate is a software project with no custom hardware), and then gives the software in detail: the technology stack, the modules, the scoring, routing and storage algorithms with their equations, flowcharts and pseudo-code, and the training of the learned scorer. It then explains the experimental design that turns the system into a measurement instrument, and closes with the chat application built on the same code and the tests that keep it correct.

3.1 Block diagram

MemGate sits between an agent and a frozen LLM and controls one thing: what goes into the prompt (Fig. 3.1). As in retrieval augmentation [14], the base model is a black box that is never trained or fine-tuned, so any difference that is measured comes from the memory policy.

The system has a *write path* and a *read path*. On the write path, every message, including the model's own replies, enters Tier 1 verbatim. When it leaves Tier 1 it is scored once by the *Adaptive Memory Decision Engine* and routed: kept as a durable fact in Tier 3, compressed to a gist in Tier 2, or dropped. After every write a budget enforcer holds the whole store at or below its budget S. On the read path, each question in a new message becomes its own retrieval query, and a context assembler packs the most relevant items into at most B tokens. The model receives only this assembled context, never the raw history.

3.2 Hardware description

MemGate is a software system and has no custom hardware, sensors or circuits. The hardware it needs is a general-purpose computer, and it was deliberately designed to run on an ordinary laptop with no GPU, so that a memory layer is practical on the kind of device an end user owns. Table 3.1 summarises the computing resources used during development and evaluation.

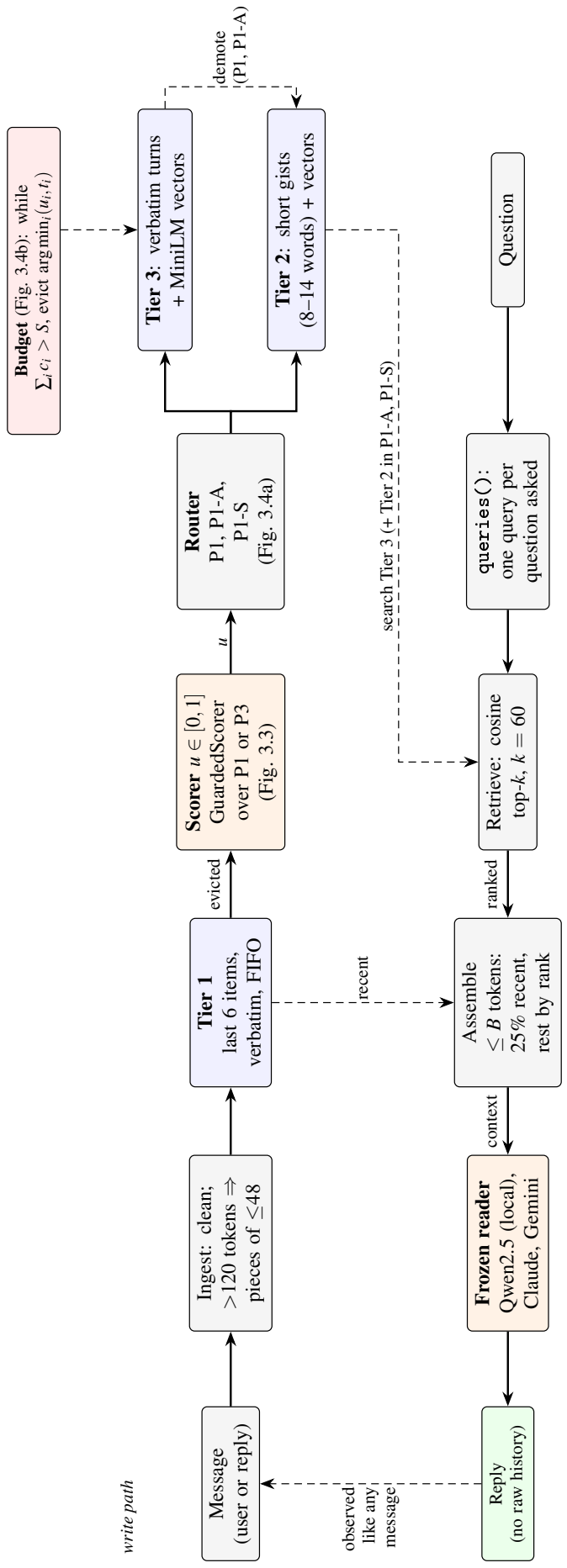


Figure 3.1. Block diagram of MemGate. Write path (top): every message enters Tier 1 verbatim; when it leaves Tier 1 it is scored once and routed, and the budget enforcer holds the store at S. Read path (bottom): each question in a message is its own retrieval query, and the reader receives only the assembled context of at most B tokens. Dashed arrows carry state.

Table 3.1. Computing resources used and required.

Resource	Use and requirement
Development laptops	Code, tests and the chat application. Python 3.10 or newer on Windows, macOS or Linux; about 4 GB of free disk for the models.
Cloud container (shared, 4 CPU cores)	Benchmark runs, scorer training and the scale and load tests. The application averaged 0.49 s per message on it while holding million-token chats.
GPU (Colab / university GPU)	Only for the LLM-judge (P2), the end-task reader and abstractive compression; about fifteen minutes of GPU time per corpus for the judge. The shipped system needs no GPU.
Apple-silicon Mac (optional)	Runs the local reply model 4-bit through `mlx`.
Network	Needed only for the first setup (model downloads) and for the optional hosted reply model; the memory layer itself runs offline.

3.3 Software description

3.3.1 Technology stack

Table 3.2 lists the software used. The core library depends only on NumPy, `tiktoken` and a sentence-embedding model; every other component is optional and degrades to a standard-library fallback that is recorded in the results, so a run can never silently change its inputs.

Table 3.2. Technology stack.

Layer	Technology
Language	Python 3 (about 15,800 lines across the library, the experiment scripts and the tests)
Numerics	NumPy; scikit-learn [51] for the learned scorer; Matplotlib for figures
Tokeniser	`tiktoken` `cl100k_base`, so every budget is an exact token count
Embeddings	all-MiniLM-L6-v2 [24, 25], 384 dimensions, through sentence-transformers or a bundled ONNX copy
Local LLMs	Qwen2.5-0.5B-Instruct (the P2 judge) and Qwen2.5-1.5B-Instruct (end-task reader and offline reply model) [52], through Hugging Face Transformers or `mlx-lm`
Hosted LLM (optional)	Google Gemini [53] through its REST API, or Anthropic Claude, for replies in the application
Documents	`pypdf` for PDF; Word and PowerPoint read with the standard library
Web application	Python standard-library HTTP server; a single-page HTML/CSS/JavaScript front end
Deployment	Docker image for a Hugging Face Space (optional online demo)
Paper	LaTeX with generated tables and TikZ diagrams
Workflow	Git and GitHub, pull requests with continuous integration (GitHub Actions running the test suites)

3.3.2 Module structure

The library is organised so that each memory decision lives in exactly one module and can be swapped without touching the others (Table 3.3). The full list of files is in Appendix C.

Table 3.3. Modules of the MemGate library (`memgate/memgate/`).

Module	Responsibility
`types.py`	Data types: memory item, turn, question (with multi-evidence and supersession links)
`utils.py`	Token counting and embeddings, with fallbacks and a record of which backend ran
`store.py`	The three-tier store: dedupe, semantic supersession, consolidation, demotion and the storage-budget loop
`scoring.py`	Scorers P0 and P1, and the interface every scorer implements
`pretrained.py`	The shipped learned scorer (P3), loaded from plain JSON weights
`judge.py`, `llm_judge.py`	The LLM judge of salience (P2)
`policies.py`	Baselines (full context, oracle, P0, RAG) and the MemGate policies: routing and context assembly
`compress.py`, `abstractive.py`	Extractive and abstractive compression of evicted turns
`external_baselines.py`	MemGPT-style and Mem0-style memory rebuilt in the same harness
`online.py`	A scorer learned online from retrieval feedback
`ingest.py`	Any text in: cleaning, chunking, file reading and scorer guards (application only)
`reader.py`	Reply models: Gemini, Claude, local Qwen, with fallback
`explain.py`	"Why this score" breakdowns for the application
`data.py`, `harness.py`	Benchmark loaders with integrity checks; metrics, sweeps and CSV output

3.3.3 Tiers as a lifecycle

The store has three tiers, but an item moves through them over its life rather than being filed into one. Every turn enters Tier 1 word for word, and the routing decision is made once, when the turn is evicted from Tier 1. With u the utility the scorer gives an item, the rule is

$$\begin{aligned} u \geq \tau_{\text{fact}} &\rightarrow \text{Tier 3: atomic fact} + \text{vector} \\ u \geq \tau_{\text{low}} &\rightarrow \text{Tier 2: compressed gist} \\ \text{otherwise} &\rightarrow \text{dropped} \end{aligned} \tag{3.1}$$

with $\tau_{\text{fact}} = 0.45$ and $\tau_{\text{low}} = 0.15$; an item with $u \geq \tau_{\text{fact}} + 0.2$ also leaves a gist in Tier 2. The tiers follow the working-set [28] and multi-store [26] views of memory. Tier 1 holds recent verbatim turns and answers *when*; Tier 2 holds compressed gists and answers *gist*; Tier 3 holds durable facts with embeddings, answers *what*, and enters the context only when retrieval matches. Because each item is scored exactly once, at eviction, the decision has a single site in the pipeline, which is what allows a controlled swap to target it.

3.3.4 Policies

Table 3.4 lists the policies ("arms") that are compared. Two need comment. RAG is the no-policy control: it stores everything, retrieves the top k items, and evicts first-in-first-out when the store is full. It is not a weak baseline but the arm that a decision policy has to beat in order to have contributed anything. P0 is given the same token budget

as MemGate, so the only difference between them is which turns are selected, not how many tokens may be spent.

Table 3.4. The policies compared.

Arm	Description
Full	Full context: the whole history, unbounded (reference only)
Oracle	Exactly the cited evidence turns (diagnostic upper bound)
P0	Sliding window: the most recent turns that fit the budget
RAG	Store all, retrieve top-k, evict oldest first (FIFO)
P1	MemGate, threshold mode: route at ingest by Eq. (3.1)
P1-A	MemGate, adaptive: score orders eviction; evicted items are compressed
P1-S	MemGate, select: score orders eviction; evicted items are dropped
P2	P1-S with a 0.5B LLM judge as the scorer
P3	P1-S with the learned scorer, never fitted on the conversation it scores

3.3.5 Scoring a message

Every equation in this section is transcribed from the code that produced the results. A message m is always scored with its speaker tag removed, and the scorer sees its text and speaker only: no question, evidence label or future turn.

The P1 heuristic. Let $W(m)$ be the words of m, D a list of 28 durability cues (*deadline, budget, prefer, born, lives, works*, ...), and $h(m) = |W(m) \cap D|$. With indicators δ_{num} (a digit), δ_{tmp} (an amount of money or a month), δ_{prop} (a capitalised word not opening a sentence), $\delta_{?}$ (ends with a question mark), δ_{asst} (written by the assistant) and δ_{upd} (a correction cue such as *actually* or *no longer* together with a digit or month),

$$\begin{aligned} s(m) &= 0.15 + \min(0.35,\, 0.18\,h) + 0.20\,\delta_{\text{num}} + 0.15\,\delta_{\text{tmp}} \\ &\quad + 0.12\,\delta_{\text{prop}} - 0.12\,\delta_{?} - 0.10\,\delta_{\text{asst}} + 0.10\,\delta_{\text{upd}}, \\ u_{\text{P1}}(m) &= \begin{cases} 0.02, & m \text{ is filler}, \\ \min\big(1, \max(0,\, s(m))\big), & \text{otherwise}, \end{cases} \end{aligned} \tag{3.2}$$

where a message is filler when it is an acknowledgement or greeting from a closed list, or has at most two words. The weights were set by hand and never fitted.

The learned scorer P3. Its input concatenates 15 hand features $\phi(m)$ (word and character counts, digit, month, money, proper noun, question, final period, $h(m)$, correction cue, filler, assistant speaker, share of words of at most three letters, capital letters, clock time) with the 384-dimensional unit-norm MiniLM embedding $e(m)$:

$$\begin{aligned} x(m) &= \big[\phi(m);\, e(m)\big] \in \mathbb{R}^{399}, \\ z(m) = b + \sum_{j=1}^{399} w_j \,\frac{x_j(m) - \mu_j}{\sigma_j}, &\qquad u_{\text{P3}}(m) = \frac{1}{1 + e^{-z(m)}}, \end{aligned} \tag{3.3}$$

with μ, σ the training mean and standard deviation of each input. The label is distant supervision [54]: $y_i = 1$ when any question cites turn i as evidence. With N training turns from corpora c grouped into families $f \in F$ (LoCoMo, REALTALK, BEAM, MemBench, everyday chat), the fit is an ℓ_2-regularised, class-balanced, sample-weighted logistic regression [51],

$$\begin{aligned} \min_{\mathbf{w},b} \ \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \omega_i \, \kappa_{y_i} \, \ell\big(y_i, u_{\mathrm{P3}}(m_i)\big), \\ \omega_i = \frac{N\,\pi_{c(i)}}{|F|\,n_{c(i)}}, \qquad \kappa_y = \frac{N}{2N_y}, \end{aligned} \tag{3.4}$$

where ℓ is the log loss, $C = 1$, n_c is the number of turns of corpus c, and π_c is the share of its family's weight that corpus c receives (by turns for the first model, by evidence turns for the larger one). Every family therefore weighs the same, whatever its size. Generated small-talk lines enter as negatives with total weight $0.05N$.

The shipped ensemble. The shipped scorer averages the logits of two such models, A (the earlier recipe) and B (all 856,876 turns). Because each logit is linear in x, the mean is again a single logistic regression, which is how it is stored (Fig. 3.2):

$$\begin{aligned} \bar{z}(m) = \tfrac{1}{2}\big(z_A(m) + z_B(m)\big) = \bar{b} + \bar{\mathbf{w}}^{\top} x(m), \\ \bar{w}_j = \tfrac{1}{2}\sum_{k\in\{A,B\}} \frac{w_j^{(k)}}{\sigma_j^{(k)}}, \quad \bar{b} = \tfrac{1}{2}\sum_k \Big(b^{(k)} - \sum_j \frac{w_j^{(k)}\mu_j^{(k)}}{\sigma_j^{(k)}}\Big). \end{aligned} \tag{3.5}$$

Because the thresholds of Eq. (3.1) read u as an absolute value, the logit is mapped by an affine function that leaves every ranking unchanged, in the manner of Platt scaling [55]:

$$z' = \alpha\bar{z} + \beta, \qquad \alpha = \frac{\operatorname{logit}\tau_{\mathrm{fact}} - \operatorname{logit}\tau_{\mathrm{low}}}{q_{\mathrm{fact}} - q_{\mathrm{low}}}, \qquad \beta = \operatorname{logit}\tau_{\mathrm{fact}} - \alpha\, q_{\mathrm{fact}}, \tag{3.6}$$

where q_{fact} and q_{low} are the quantiles of $\bar{z}$ over LoCoMo and REALTALK turns that put 36.48% of them above τ_{fact} and 71.85% above τ_{low}, the shares of the first shipped model.

The application's guard. The application wraps its scorer in a *GuardedScorer* (Fig. 3.3). Each guard fires on one kind of input the scorers were not built for (no words, small talk, code, non-Latin script, repetition), and ordinary English passes straight through. One further rule corrects a measured weakness of the learned scorer, which gave a short dated fact the score of a greeting: for a message of $n(m) \le 30$ words that names a month, a clock time or an amount,

$$u_{\mathrm{G}}(m) = \max\big(u_{\mathrm{P3}}(m), u_{\mathrm{P1}}(m)\big), \tag{3.7}$$

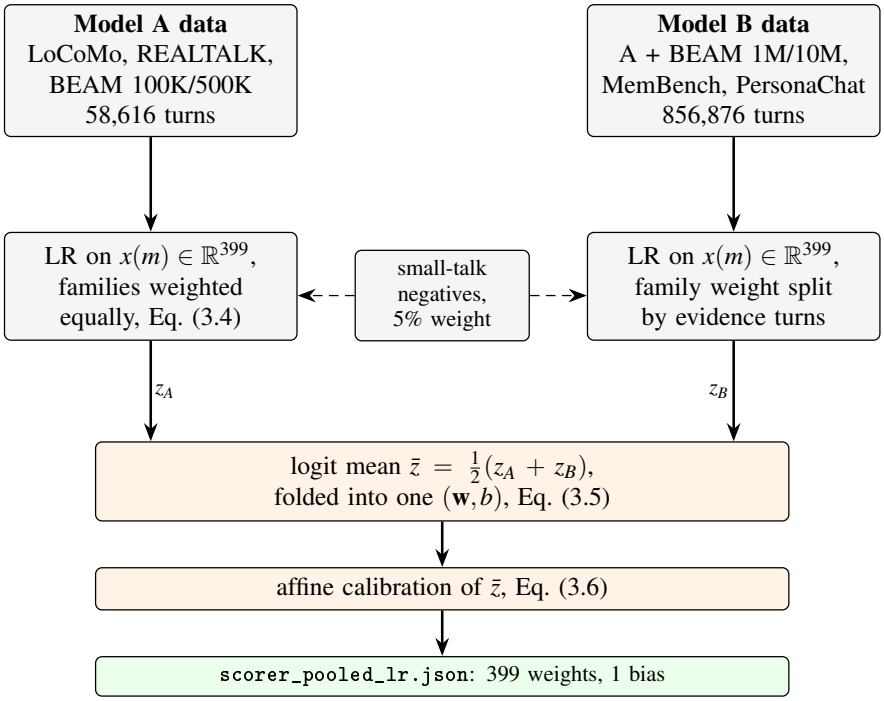


Figure 3.2. How the shipped scorer is built: two logistic regressions, Eq. (3.4), averaged in logit space and folded into one, Eq. (3.5), then calibrated, Eq. (3.6). It ships as 399 weights and a bias in plain JSON.

and $u_{\mathrm{G}} = u_{\mathrm{P3}}$ otherwise. The experiments never use the guard.

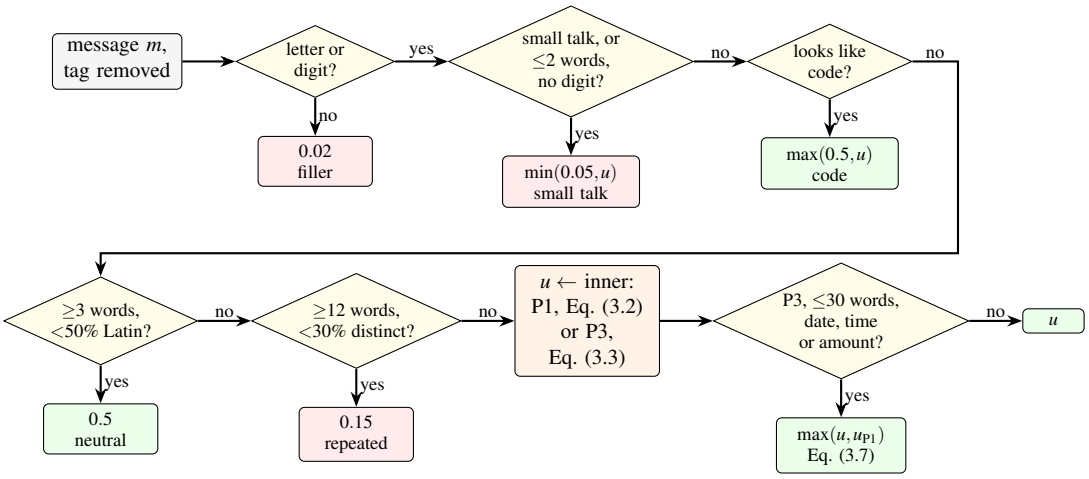


Figure 3.3. How the application scores a message (GuardedScorer). Guards are tried left to right; a message that none of them covers is scored by the inner scorer, and a short message naming a date, time or amount is floored at the P1 score, Eq. (3.7).

3.3.6 The storage budget and eviction

Each stored item i has a cost

$$c_i = \begin{cases} \mathrm{tok}(i) & \text{(budget in tokens)}, \\ \mathrm{bytes}(i) + 1536 \cdot [i \text{ has a vector}] & \text{(budget in bytes)}, \end{cases} \tag{3.8}$$

and after every write the store enforces

$$\sum_{i \in T_1 \cup T_2 \cup T_3} c_i \;\le\; S \tag{3.9}$$

by evicting, while it is violated,

$$v = \arg\min_{i \in T_3} \,(u_i, t_i) \tag{3.10}$$

in lexicographic order, t_i being the turn index, so the oldest item breaks a tie (Fig. 3.4b). Superseded records are reclaimed before any eviction. P1 and P1-A demote v to a Tier 2 gist of at most eight words when that gist costs less than v, and drop it otherwise; P1-S always drops it; when Tier 3 is empty the same rule is applied to Tier 2. Tier 1, the six-item live buffer, is never evicted. RAG uses $\arg\min_i t_i$, first in first out, and has no scorer. With equal costs, repeated application of Eq. (3.10) leaves P1-S holding exactly the highest-utility items seen so far, and RAG the newest ones, the FIFO rule of page replacement [27]; this is the only difference between them.

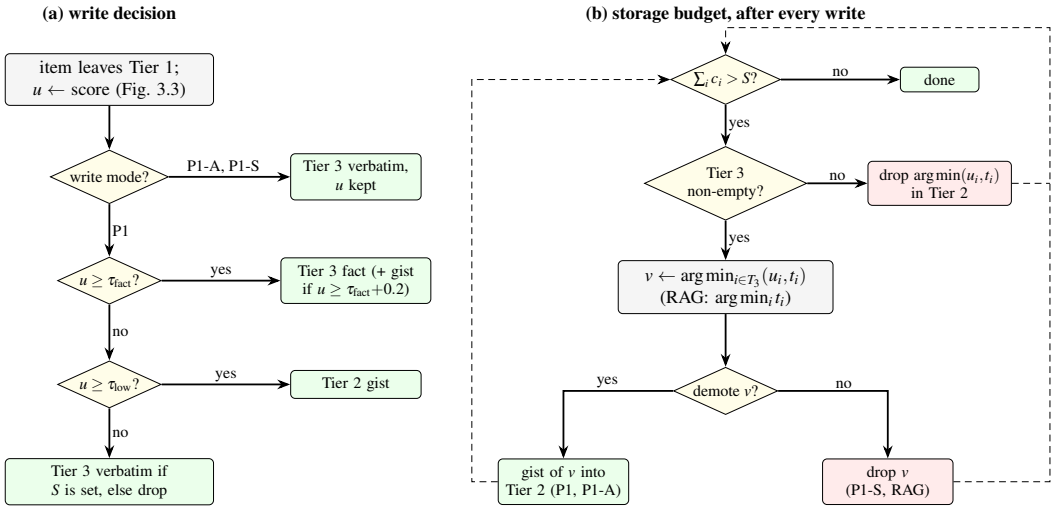


Figure 3.4. Where the keep, compress and forget decisions are taken. (a) Routing of an item leaving Tier 1, Eq. (3.1): threshold mode (P1) decides at ingest; the adaptive modes (P1-A, P1-S) keep everything verbatim and let u order the eviction queue. (b) The budget loop of Eqs. (3.9)–(3.10), run after every write.

3.3.7 Context assembly and store maintenance

For a question q and context budget B (2048 tokens in every experiment), the most recent Tier 1 items are packed into $\lfloor B/4 \rfloor$ tokens. In P1-A and P1-S the remainder is filled greedily from one pool of Tier 2 and Tier 3 items,

$$R_k(q) = \operatorname{top}_k \big\{\, i : \cos\big(e(q), e(i)\big) > 0 \,\big\}, \qquad k = 60, \tag{3.11}$$

ranked by cosine similarity. P1 instead gives Tier 2 a fixed quarter of B, packed by information density, and fills the last half from Tier 3 by Eq. (3.11). When a message asks several questions, each is its own query and the rankings are interleaved, best of each first.

Three rules keep the store consistent, and none of them reads an evaluation label. A write to Tier 3 whose embedding has $\cos \geq 0.92$ with an active fact is folded into that fact instead of being stored twice. A write that the P1 heuristic labels a correction retires the single most similar active fact with $\cos \geq 0.60$. When Tier 2 outgrows its share, its oldest half is consolidated: items with $u \geq 0.5$ are promoted to Tier 3 verbatim and the rest are merged into one coarser gist.

3.3.8 Algorithms

Algorithms 1–3 give the three core procedures in pseudo-code. Flowcharts of the application's message and question paths are in Appendix A.

Algorithm 1 Write path: OBSERVE(m) for policies P1, P1-A and P1-S

append m to Tier 1 ▷ verbatim, live buffer of 6 items
if Tier 1 holds more than 6 items **then**
    $x \leftarrow$ oldest item of Tier 1; remove it from Tier 1
    $u \leftarrow$ SCORE(x) ▷ P1, P2 or P3; text and speaker only
    if mode is adaptive (P1-A, P1-S) **then**
        add x to Tier 3 verbatim with its utility u
    else if $u \geq \tau_{\text{fact}}$ **then**
        add x to Tier 3 as a fact; **if** $u \geq \tau_{\text{fact}} + 0.2$ **then** add its gist to Tier 2
    else if $u \geq \tau_{\text{low}}$ **then**
        add the gist of x to Tier 2
    else
        drop x
    end if
    ENFORCEBUDGET()
end if

3.4 Experimental design

3.4.1 One variable per comparison

The experimental design lies in the set of contrasts rather than the set of policies. Each contrast holds everything constant except one factor (Table 3.5). The whole study leans on the second contrast. RAG and P1-S hold the same verbatim turns, in the same embedding space, retrieved the same way, under the same storage cap. They differ only in which turns they throw out when the cap binds: the oldest for RAG, the lowest-utility for P1-S. Whatever gap opens between them belongs to the decision policy.

Algorithm 2 EnforceBudget(): hold the store at or below S

remove superseded records from Tier 3
consolidate Tier 2 if it has outgrown its share
while $\sum_i c_i > S$ **do** ▷ Eq. (3.9)
    if Tier 3 is not empty **then**
        $v \leftarrow \arg\min_{i \in T_3}(u_i, t_i)$; remove v ▷ Eq. (3.10); RAG uses $\arg\min t_i$
        if policy compresses (P1, P1-A) **and** the gist of v costs less than v **then**
            add the gist of v to Tier 2
        end if
    else if Tier 2 is not empty **then**
        remove $\arg\min_{i \in T_2}(u_i, t_i)$
    else
        break ▷ only Tier 1 is left; it is never evicted
    end if
end while

Algorithm 3 Read path: BuildContext(message, B)

$Q \leftarrow$ the questions in the message (the whole message if it has none)
$C \leftarrow$ most recent Tier 1 items that fit in $\lfloor B/4 \rfloor$ tokens
for each $q \in Q$ **do**
    $L_q \leftarrow R_{60}(q)$ over Tier 2 $\cup$ Tier 3, ranked by $\cos(e(q), e(i))$ ▷ Eq. (3.11)
end for
interleave the lists L_q, best of each first, skipping duplicates
add items to C in that order while they fit in B tokens
return C ▷ the reader sees only C

Table 3.5. Controlled contrasts: each changes exactly one factor.

Contrast	Held constant	Varies
P0 vs. RAG	retention, storage	read path
RAG vs. P1-S	storage, fidelity, retrieval	**what is forgotten**
P1-S vs. P1-A	storage, selection	**fidelity** (drop vs. compress)
P1 vs. P2 vs. P3	everything but the scorer	**the scorer**

3.4.2 Evaluation metrics

Two retrieval-level metrics are used. *Evidence recall* (strict recall) is the share of questions for which every gold evidence turn made it into the assembled context. *Answer recall* is the share for which the answer text itself survived into the context, a string-containment test close in spirit to extractive QA scoring [34], computed over the 715 of 1,527 LoCoMo questions whose answer can be read directly off the evidence turns.

The difference between the two matters. Evidence recall credits an identifier: a compressed gist keeps the identifier of a turn whose text it threw away, so it scores a hit with none of the answer behind it. During development, evidence recall climbed from 8.1% to 14.8% while answer recall stayed at 12.8%; the whole gain was bookkeeping. Answer recall is therefore the primary metric, and evidence recall is reported next to it so that any disagreement stays visible. Both measure context recall, which bounds end-task accuracy from above; Section 4.9 measures accuracy directly through a reader model.

3.4.3 Datasets and set-up

The main corpus is LoCoMo [29]: 10 multi-session conversations, 5,882 turns and 1,527 scoreable questions. Category-5 (adversarial) items are left out, since the right response is to decline and retrieving their evidence would not count as success. REALTALK, BEAM, MemBench, PersonaChat and LongMemEval are used for transfer and scale tests; their details are in Appendix B.

Retrieval and semantic supersession use all-MiniLM-L6-v2 at 384 dimensions. Budgets are counted with `cl100k_base`. The P2 judge is Qwen2.5-0.5B-Instruct [52], run locally; rather than generating a rating and parsing it, salience is read as the expected value over the digit-token logits of a single forward pass, which is faster, cannot fail to parse, and leaves no ties in the eviction order.

Every interval is a paired bootstrap [37] with 10,000 resamples drawn by conversation, paired with an exact McNemar test [35, 36]. The resampling unit matters: the 1,527 questions come from 10 conversations, so they are not 1,527 independent trials [38], and one claim does not survive the correction (Section 4.3). Every model is a pinned local copy, with the model hub disabled at import, and every numeric table is generated from committed result files rather than typed in by hand.

3.4.4 Two defects found and fixed

Two defects were found during the project, and how they were found is part of the method.

An evaluation leak. The first prototype, shown at Review 1, reported 95% recall

for MemGate against 10% for a sliding window on a synthetic set. In August 2026 it was found that an early consolidation rule promoted items using `fact_id`, which is the evaluation label. The leak was worth 45–47 recall points; the honest synthetic figure was 50%, not 95%. The number was withdrawn, the leak was fixed, and three regression tests now require the store to come out bit-identical when every label is stripped, including on the eviction and demotion paths added later. This is the classical leakage failure described in [39, 40].

A scorer defect. In October 2026 it was found that the heuristic scored each turn with its speaker tag still attached, which stopped the filler rule from firing and gave almost every turn the proper-noun bonus: 5,863 of 5,882 LoCoMo turns were labelled facts. The recall tables use the fixed scorer. The judge, end-task, abstractive and LongMemEval runs were made before the fix and are marked as such. Fixing it moved the headline P1-S result from 36.8% to 37.5%.

3.5 The chat application

To show that the memory layer works as a product and not only as an experiment, the project includes a chat application built on exactly the same `observe()` and `build_context()` calls that the experiments use. A user chats in a browser; every message goes through MemGate; and the reply model sees only the notes MemGate picks.

3.5.1 Features

- **Visible decisions.** Every message is tagged with the tier it went to and its score (for example “Tier 3 · fact · score 0.67” or “forgotten · small talk”). Clicking a tag shows which features pushed the score up or down (Fig. 3.5).
- **Memory panel.** Tabs show what each tier holds, every keep, compress and forget decision with its score, and the exact notes sent to the model for the last reply (Fig. 3.6).
- **Live comparison.** The Compare tab answers the last question from three memories at the same budgets, MemGate, a sliding window and RAG, so the difference can be seen on the user’s own chat (Fig. 3.7).
- **Entry gate.** Filler such as “hi” or “lol nice” is forgotten on arrival; a clear fact goes to Tier 3 at once.
- **Any input.** Long pastes, PDF, Word and PowerPoint files, other languages, emoji and code are cleaned, cut into pieces of at most 48 tokens, scored and stored; each recalled piece shows the file it came from (Fig. 3.8).

- **Reply models.** Google Gemini through its free API key, with retries and backup models when it is busy, or Anthropic Claude; a local Qwen2.5-1.5B model answers offline or when the hosted model is slow, so a reply never waits more than about 20 s.
- **Saved chats.** Chats are saved on the user's machine and come back after a restart; a sidebar lists them.
- **Benchmark check.** A button re-runs the LoCoMo comparison inside the app and checks it against the published numbers.
- **Online option.** A Docker image deploys the app as a Hugging Face Space with a passcode and a separate memory for each visitor.

One-command scripts (`setup.sh` / `setup.ps1` and `start.sh` / `start.bat`) create a virtual environment, install the packages, download the models and open the app at `http://localhost:8765` on Windows, macOS or Linux.

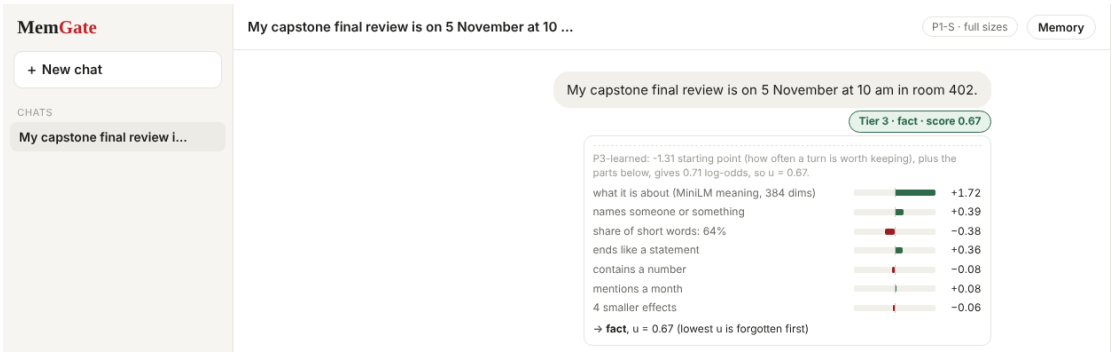


Figure 3.5. The application tags each message with its tier and score; the breakdown shows why the learned scorer kept a dated fact.

3.5.2 Server

The server is a small Python standard-library HTTP server with a JSON API: `/api/chat` sends a message, `/api/upload` adds a file, `/api/state` returns the memory panel, `/api/why` explains a score, `/api/rival` runs the comparison, `/api/bench` the benchmark check, and `/api/chats/*` manages saved chats. Request bodies are size-limited, and the accept queue was raised to 256 after a load test (Section 4.13).

3.6 Testing and verification

Every claim in this report is produced by a script in the repository and is pinned by a regression test. The test suites cover the library (tiers, compression, budgets and label-blindness), the scorers, the memory policy, the reply models, the application endpoints, input handling, the score explanations and comparison, the rebuilt MemGPT-style and

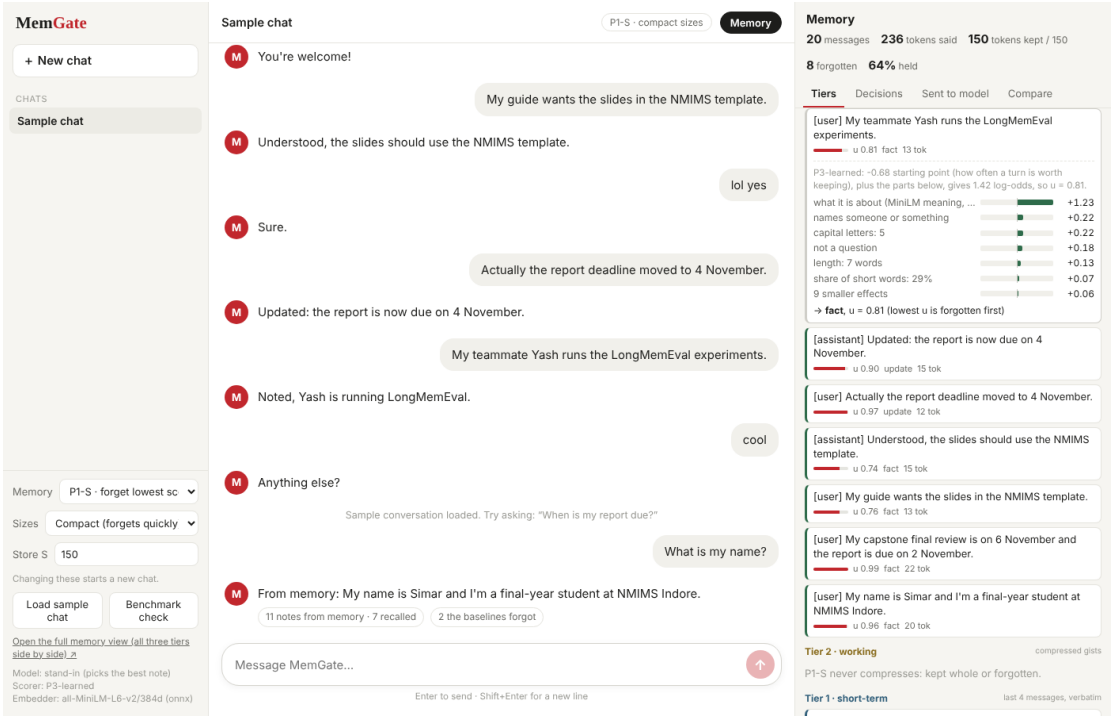


Figure 3.6. The memory panel: what each tier holds, with the score breakdown of a stored note.

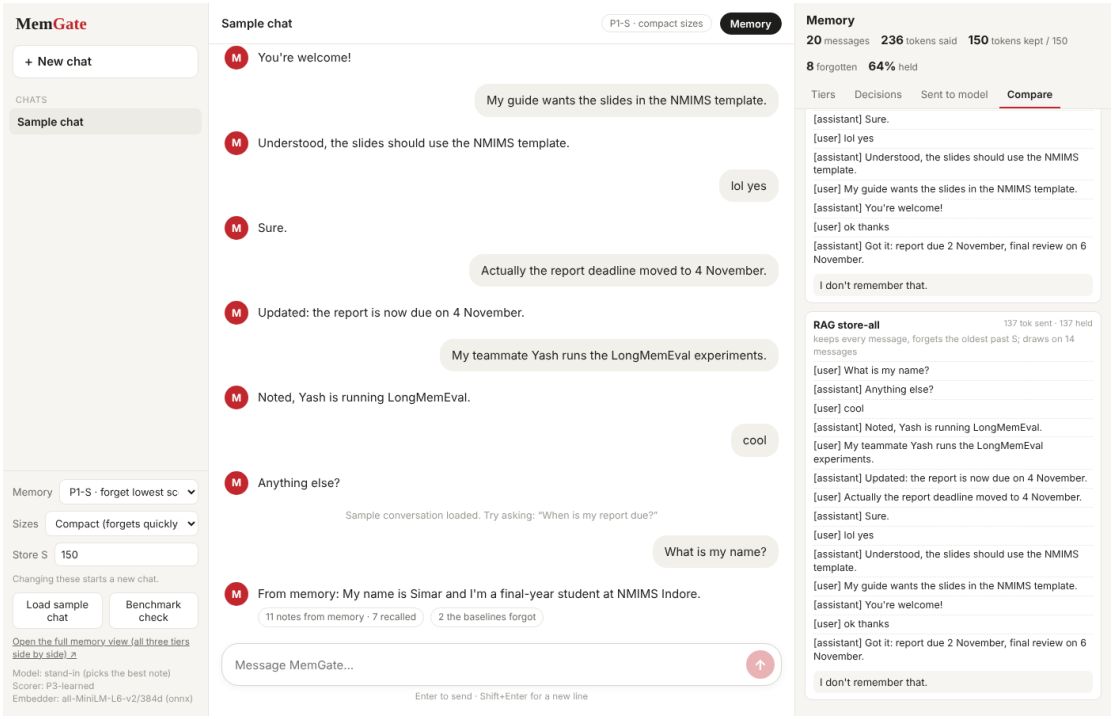


Figure 3.7. The Compare tab: the same question answered from MemGate, a sliding window and RAG at the same budgets.

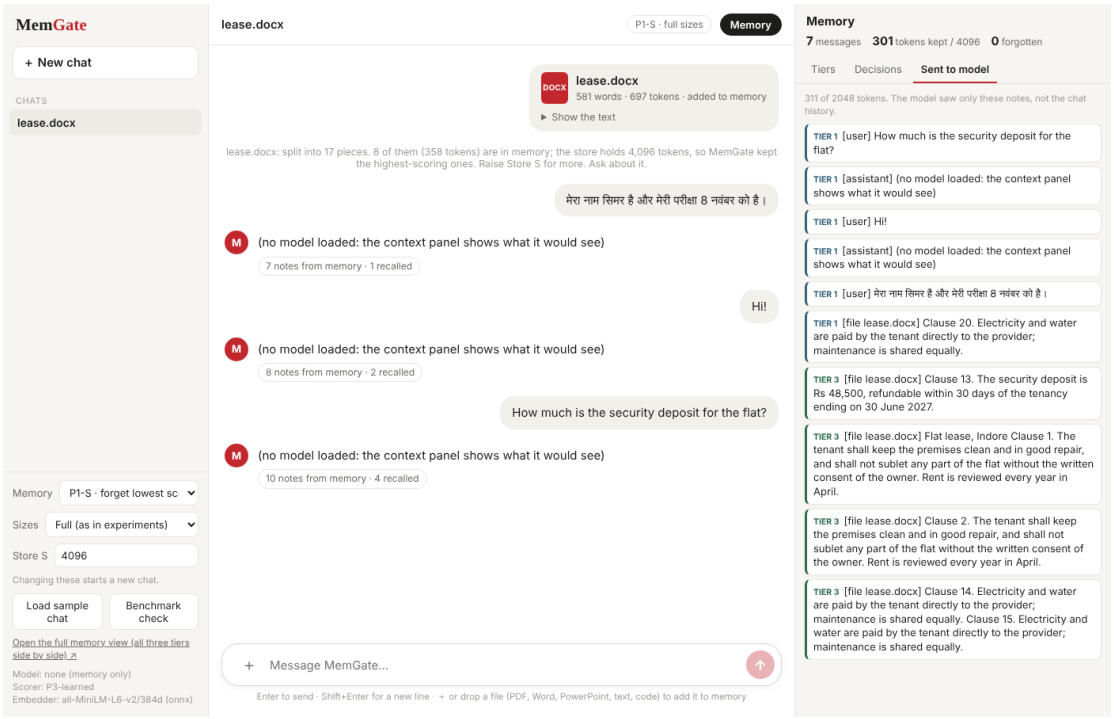


Figure 3.8. A Word file uploaded into a chat (run without a reply model): the file is cut into pieces, and the notes sent to the model show the recalled clauses with their source. A Hindi message is stored the same way.

Mem0-style baselines, and a benchmark check that replays LoCoMo and compares it with the published numbers. The last full run before this report passed all 453 tests. A GitHub Actions workflow runs every suite, with the real MiniLM embedder and the LoCoMo data, on each push and pull request.

Summary

MemGate is a pure-software memory layer with a three-tier store, a single scoring site, a hard storage budget and a retrieval-based read path. Its policies differ in exactly one factor each, so that the experiments isolate one decision at a time. The learned scorer is a 399-input logistic regression shipped as plain JSON. The same code runs a chat application that makes every memory decision visible, and the whole system is held in place by over 450 automated tests.

Chapter 4

Results and Analysis

This chapter reports what the experiments found. Each section takes one question, states the comparison that answers it, and gives the result with a 95% confidence interval resampled by conversation. The chapter moves from the accounting question (what should be budgeted), through the core decision (select or compress), the choice of scorer and its transfer, to end-task accuracy, comparison with existing memory designs, scale, and finally the behaviour of the complete application. It closes with the standards that were followed to make the results valid and reproducible. Unless stated otherwise, the context budget is $B = 2048$ tokens and answer recall is the primary metric (Section 3.4.2).

4.1 Storage was never budgeted

Work in this area caps the context assembled for each query and leaves the store unbounded, and at first so did this project. Under that accounting a store-everything baseline pays nothing to hold all 419 turns of a conversation, so forgetting can only ever lose information. When our own ablation was run that way, it reported the tiering and compression machinery as a net loss against keeping everything and retrieving. Given how cost was being counted, that verdict was correct.

A storage budget S changes the question from what fits in the context to what is worth keeping at all. Table 4.1 sweeps S with the context budget fixed at 2048 tokens, and the ranking under this accounting is not the ranking under the old one. This is the most consequential observation of the project: a protocol that leaves out a cost ends up answering a different question from the one it seems to ask.

4.2 Selection pays; compression does not

Table 4.1 reports both metrics across the sweep, and Fig. 4.1 plots them. P1-S, which drops the lowest-utility item when the cap binds, leads on answer recall at every budget. P1-A, which compresses that item instead, leads on evidence recall at every budget, and yet at $S = 8192$ it falls behind even RAG on answer recall, by 3.4 points.

Here the identifier problem of Section 3.4.2 shows up in real numbers. A demoted gist keeps the evidence identifiers of a turn whose text is gone. Judged on evidence

recall alone, compression looks like the better use of a bounded store; checked for whether the answer text survived, it looks like the worse one.

Table 4.1. Recall against the storage budget S in tokens, context budget 2048. Δ is P1-S minus RAG on answer recall: identical storage, fidelity and retrieval, differing only in which turns are forgotten.

	Evidence recall (%)			Answer recall (%)			
S	RAG	P1-A	P1-S	RAG	P1-A	P1-S	Δ
1024	4.3	9.3	5.3	9.1	11.9	**15.8**	+6.7
2048	10.1	19.4	11.8	18.3	22.7	**27.8**	+9.5
4096	17.8	31.8	20.2	28.1	30.3	**37.5**	+9.4
8192	30.6	45.6	38.6	44.3	41.0	**57.8**	+13.4
16384	53.3	58.2	57.9	66.4	63.1	**71.3**	+4.9

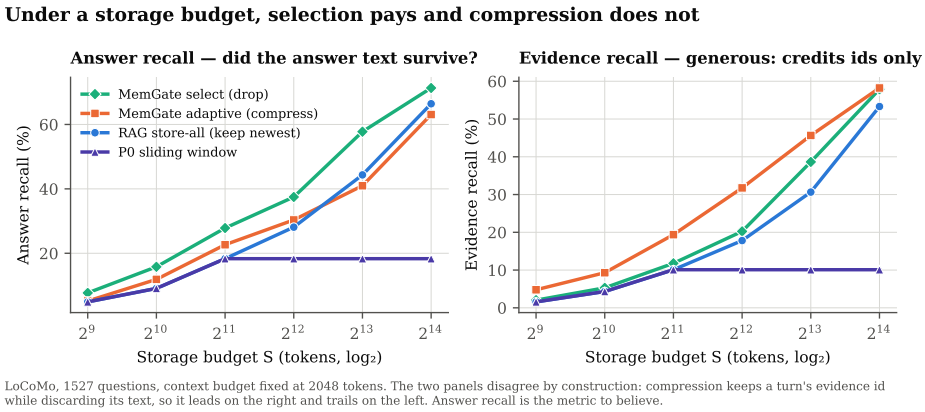


Figure 4.1. Recall against the storage budget for RAG, P1-A (compress) and P1-S (drop).

4.3 Statistical significance

Table 4.2 gives each contrast with intervals resampled by conversation. Scored eviction beats FIFO on answer recall by +9.37 points, and the interval excludes zero. The matching evidence-recall gain of +2.42 points does not survive clustering ($p = 0.08$), so it is not claimed. It is still reported, because a claim that fails a correctness check says something useful about the check. The P1-S versus P1-A rows put signs on the divergence: +7.13 on answer recall and −11.53 on evidence recall, both significant. The two metrics disagree about direction, not just size.

4.4 The rate–distortion crossover is not reached

Rate–distortion reasoning says compression must win eventually: compress a record of everything far enough and it should beat a faithful record of only part of it [20]. Shrinking S cannot test this, because every policy hits the floor first, near 0.2% evidence recall at $S = 256$. The missing axis is length, so up to ten LoCoMo conversations (186K tokens) were concatenated with S fixed, raising the compression ratio to 182×.

Table 4.2. Paired bootstrap (10,000 resamples) by conversation, with exact McNemar, at $S = 4096$. The second row is the claim we decline to make.

Comparison	Metric	Δ (pts)	95% CI (clustered)	McNemar p	
P1-S vs. RAG (FIFO)	answer	$+9.37$	$[+5.45, +14.90]$	2.8×10^{-5}	✓
P1-S vs. RAG (FIFO)	evidence	$+2.42$	$[-1.48, +6.43]$	0.08	N.S.
P1-S vs. P1-A (compress)	answer	$+7.13$	$[+4.32, +10.28]$	2.1×10^{-5}	✓
P1-S vs. P1-A (compress)	evidence	-11.53	$[-14.73, -8.47]$	1.8×10^{-23}	✓
P1-S vs. P0 (recency)	answer	$+19.16$	$[+15.42, +24.13]$	8.2×10^{-19}	✓
P1-S vs. P0 (recency)	evidence	$+10.15$	$[+6.58, +13.84]$	1.2×10^{-15}	✓

Table 4.3 and Fig. 4.2 show no crossover: the margin of selection over compression narrows but stays positive, and the best operating point stays at the vertex, a subset kept at full fidelity.

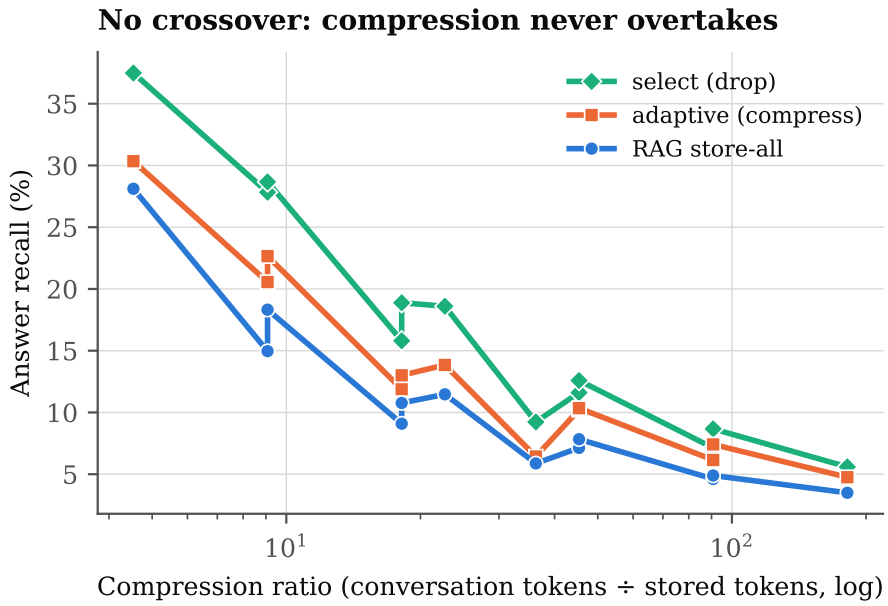
A concatenated stream is not a natural long conversation: each question is about one piece and the rest act as distractors. This is therefore a stress test of retention rather than a long-dialogue benchmark, and the crossover is ruled out only along this axis and up to this ratio.

Table 4.3. Answer recall (%) against compression ratio at $S = 2048$. The last column is the margin of dropping over compressing.

Ratio	RAG	P1-A	P1-S	P1-S − P1-A
9×	18.3	22.7	**27.8**	+5.2
18×	10.8	13.0	**18.9**	+5.9
45×	7.8	10.3	**11.6**	+1.3
91×	4.6	7.4	**8.7**	+1.3

4.5 The embedding index is not free

Counting storage in text tokens ignores the index that makes retrieval possible. A 384-dimensional `float32` vector occupies 1,536 B, while a LoCoMo turn averages about 130 B of text, so the index outweighs the content it indexes by roughly a factor of 12, and the limiting resource becomes the number of stored items rather than their length. Table 4.4 and Fig. 4.3 repeat the sweep with the budget in bytes and each retrievable item charged for its vector. The selection result survives the change of units: P1-S leads at every budget above 32 KiB. Below about 40 KiB, however, an embedding index cannot pay for its own storage, so P0, which keeps no index, does best. A protocol that counts only text never sees this regime, yet it is exactly where a memory layer on a small device would live.



Streams of 1–10 concatenated LoCoMo conversations at three storage budgets. The rate–distortion prediction is that compression must win once the store is tight enough; it does not, up to 182×.

Figure 4.2. Answer recall as the compression ratio grows to 182×.

Table 4.4. Answer recall (%) with the budget in bytes, charging 1,536 B per retrievable item for its embedding. P0 keeps no index and is flat.

Budget (KiB)	P0	RAG	P1-A	P1-S
32	**18.3**	5.0	5.9	15.1
64	18.3	11.0	9.4	**24.9**
128	18.3	20.0	11.9	**33.1**
256	18.3	33.0	15.8	**46.0**
512	18.3	50.3	22.2	**61.0**

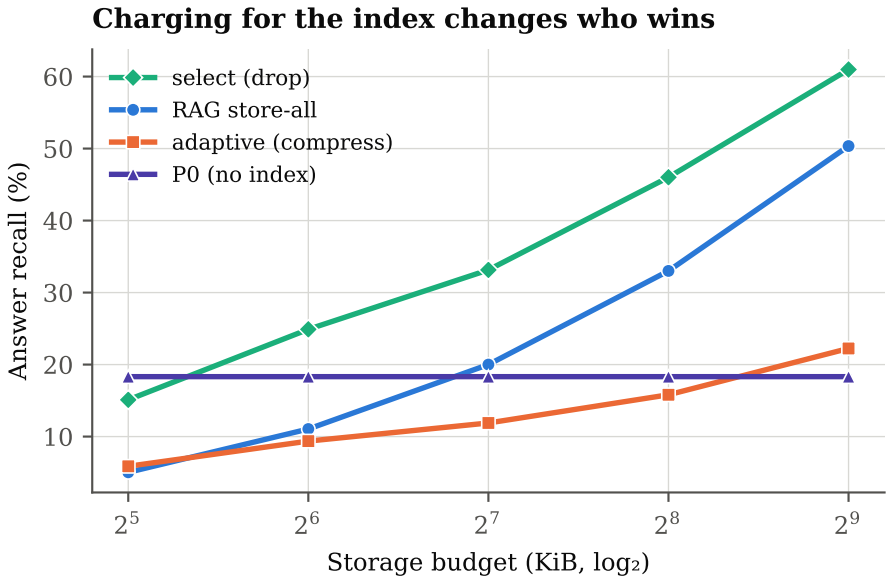


Figure 4.3. Answer recall with the storage budget counted in bytes, the index included.

4.6 Scorer analysis

If the value lies in the decision, the scorer that makes it is the part worth improving. Table 4.5 removes one mechanism at a time. Retrieval dominates, at −31.6 evidence points, and scored eviction is worth 6.4 points. The demotion row shows the same divergence in miniature: removing demotion costs 11.5 evidence points and gains 7.1 answer points.

Table 4.5. Component ablation at $S = 4096$: change from the full system. †: the row on which the two metrics disagree in direction.

Component removed	Δ evidence	Δ answer
Retrieval (Tier 3)	−31.6	−28.0
Demotion on pressure †	−11.5	+7.1
Scored eviction (→ FIFO)	−6.4	−7.4
Informative compressor	−4.4	−1.7
Consolidation merge	−3.4	−4.8
Semantic supersession	−0.3	−0.3

Table 4.6 and Fig. 4.4 vary only the scorer. Three results follow, all significant under clustering. Scoring is worth doing: the hand-written heuristic gains +11.19 answer points over unscored recency. The LLM judge [33], however, loses: Qwen2.5-0.5B reaches 30.9% against the heuristic's 36.8%, −5.87 points with the interval excluding zero, at about fifteen minutes of GPU time per corpus against microseconds for a regular expression. And the headroom is real: a logistic scorer fitted leave-one-conversation-out reaches 48.3%, +11.47 over the heuristic.

The judge fails because of what it is shown. It rates each turn on its own, but salience is not a property of a turn on its own: a phone number matters because someone asks for it later. The fitted scorer sees the same isolated text but absorbs corpus-level base rates that a per-turn prompt cannot convey; its strongest single feature is turn length, ahead of every hand-designed cue.

Table 4.6. Single-component scorer swap at $S = 4096$. Policy, retrieval and compression fixed; only the scorer varies. P3 is fitted on the other nine conversations. Run before the scorer fix (Section 3.4.4).

	What the scorer sees	Evid.	Ans.	Δ vs. P1	95% CI
P0	position only	12.6	25.6	−11.19	[−17.24, −6.19]
P1	regex + shallow NER	19.8	**36.8**	—	—
P2	the turn, in isolation	14.0	30.9	−5.87	[−10.77, −1.76]
P3	turn + fitted base rates	33.4	48.3	+11.47	[+4.90, +17.86]

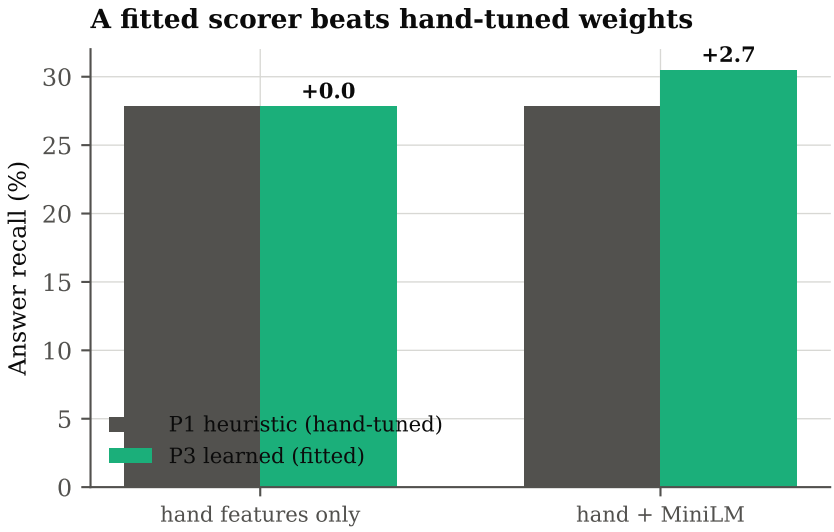


Figure 4.4. Answer recall by scorer: recency, heuristic, LLM judge and learned.

4.7 A learned scorer across two corpora

LoCoMo's dialogues are generated by an LLM, so a scorer fitted to them may learn the generator's style rather than what people later ask about. This was tested on REALTALK [30]: ten conversations between real people over 21 days, 8,944 turns and 660 questions, of which 160 are answer-recoverable. Every held-out conversation is scored by three models that never saw it: trained on the other conversations of the same corpus, on the other corpus only, or on all 19 other conversations pooled.

Table 4.7 reports the result. On LoCoMo the pooled scorer reaches 52.0% answer recall against the heuristic's 37.5%, a gain of +14.55 points, and pooling beats same-corpus training by +3.36, so adding a different corpus helped rather than diluted. Trans-

fer works in one direction only. Trained on REALTALK alone, the scorer still beats the heuristic on LoCoMo by +9.09; trained on LoCoMo alone, it falls below the heuristic on REALTALK (−4.38, n.s.) and is significantly worse than same-corpus training there.

Because $S = 4096$ is the budget of the headline comparison rather than one chosen for this scorer, Table 4.8 repeats the pooled run at four other budgets. On LoCoMo the gain is significant from $S = 2048$ to 16384 and largest at 4096. On REALTALK it is significant at $S = 8192$ (+16.88) and 16384. The embedding is what carries across: with the hand features alone, the pooled scorer is significantly worse than the heuristic on REALTALK (−8.75), because its feature weights encode LoCoMo's style. Logistic regression was kept after comparing it with boosted trees and a 64-unit MLP: neither beat it by more than the spread between held-out conversations, and the trees were much slower.

Table 4.7. Learned scorer on held-out conversations of two corpora, $S = 4096$. Δ is answer recall against P1 with 95% intervals by conversation; bold excludes zero.

Scorer (trained on)	AUC	Evid.	Ans.	Δ vs. P1	95% CI
LoCoMo (715 answer-recoverable questions)					
RAG (FIFO, no scorer)	–	17.8	28.1	—	—
P1 heuristic	0.599	20.2	37.5	—	—
P3, same corpus	0.793	32.9	48.7	+11.19	[+4.72, +17.06]
P3, other corpus only	0.692	30.1	46.6	+9.09	[+4.53, +13.46]
P3, pooled (shipped)	0.791	34.0	52.0	+14.55	[+10.40, +19.70]
REALTALK (160 answer-recoverable questions)					
RAG (FIFO, no scorer)	–	10.5	16.9	—	—
P1 heuristic	0.624	12.6	32.5	—	—
P3, same corpus	0.737	18.5	38.8	+6.25	[−3.01, +13.42]
P3, other corpus only	0.681	11.2	28.1	−4.38	[−12.50, 0.00]
P3, pooled (shipped)	0.749	17.3	36.2	+3.75	[−2.92, +8.65]

Table 4.8. The pooled learned scorer (P3) against the P1 heuristic across the store budget S, answer recall (%). Bold excludes zero.

	LoCoMo			REALTALK		
S	P1	P3	Δ [95% CI]	P1	P3	Δ [95% CI]
1024	15.8	19.0	+3.22 [0.00, +6.40]	8.1	10.0	+1.88 [−1.95, +5.36]
2048	27.8	33.7	+5.87 [+1.83, +9.88]	16.2	15.0	−1.25 [−10.62, +3.98]
4096	37.5	52.0	+14.55 [+10.40, +19.70]	32.5	36.2	+3.75 [−2.92, +8.65]
8192	57.8	69.1	+11.33 [+6.06, +16.41]	41.2	58.1	+16.88 [+6.31, +26.72]
16384	71.3	77.2	+5.87 [+2.04, +10.56]	61.3	68.1	+6.88 [+2.12, +12.21]

4.8 A negative result on LongMemEval

The headline was also tested on LongMemEval [16]: 94 independent histories of 38–62 sessions each, with the store budget swept so that LongMemEval was held at LoCoMo's retention (22.1% against 22.0%). Here the heuristic P1-S *loses* to FIFO by

21.0 answer points (95% CI $[-33.9, -8.1]$), where LoCoMo gives a gain. The cause is the scorer rather than the mechanism: the heuristic's separation between evidence and non-evidence turns reverses sign, from $+0.0476$ on LoCoMo to -0.0346 on LongMemEval, so eviction discards evidence first. The heuristic was written for LoCoMo's short dialogue lines, and LongMemEval's turns are long prose. An obvious confound, the share of the context reserved for recent turns, was ruled out: removing it changed answer recall by exactly zero. This run used the heuristic before the scorer fix; the learned scorer has not yet been tested on LongMemEval, and that is listed as future work.

4.9 End-task accuracy

Everything so far is context recall, which caps end-task accuracy without measuring it. To measure it, each assembled context was passed to one fixed reader, Qwen2.5-1.5B-Instruct-4bit decoding greedily for at most 32 tokens, and the output was scored with the same normaliser the answer-recall metric uses, over all 715 answer-recoverable questions, with a closed-book arm (no memory) and an oracle arm (exactly the cited evidence).

Table 4.9 and Fig. 4.5 report the outcome. The closed-book floor is 1.7%, so the margin above it is due to the memory layer. P1-S reaches 18.5% against RAG's 12.4%, a realised gap of $+6.0$ points (95% CI $[+3.3, +9.1]$), which is 69% of the $+8.7$-point recall gap between the same two arms in this run. Accuracy given that the answer was present is 49.4% for P1-S against 42.3% for RAG, so P1-S both places the answer in context more often and assembles contexts the reader uses more successfully. The oracle converts a complete context into a correct answer 67.3% of the time, which locates the remaining headroom in the reader rather than in the memory layer.

Table 4.9. End-task accuracy through one fixed reader at $S = 4096$ over all 715 answer-recoverable questions. Acc.|present is accuracy on the questions whose answer reached the context. Run before the scorer fix.

Arm	Acc.	F1	Ans. present	Acc.|present
Closed-book *(floor)*	1.7	5.0	0.0	–
P0 sliding window	8.3	11.9	18.3	40.5
RAG store-all	12.4	17.7	28.1	42.3
P1-A adaptive (compress)	13.1	19.9	30.1	42.3
P1-S select (drop)	**18.5**	24.7	36.8	49.4
Oracle *(ceiling)*	67.3	75.7	100.0	67.3

Two further runs refine the compression result. Replacing the extractive compressor with the 1.5B model rewriting each evicted turn gains $+4.06$ answer points over extractive compression (95% CI $[+0.84, +6.66]$), so part of the compression loss was the compressor; dropping still wins, 36.8% against 34.1%, at about 1,850 times the

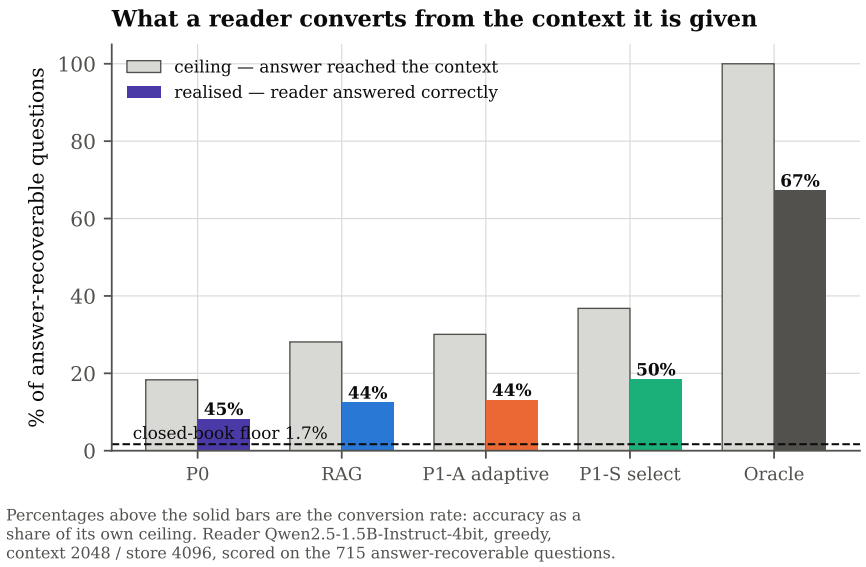


Figure 4.5. Reader accuracy by memory policy, with the closed-book floor and the oracle ceiling.

compute. A scorer learned online from retrieval feedback, with no labels and no future questions, is statistically level with the heuristic (-2.94, CI $[-7.93, +1.13]$).

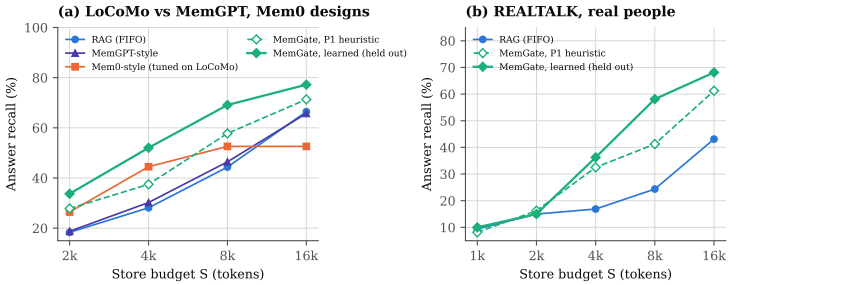
4.10 Against MemGPT-style and Mem0-style memory

To compare MemGate with designs in actual use, the memory management of MemGPT [10] and Mem0 [13] was rebuilt inside the same harness at the same budgets. The MemGPT-style arm keeps a recent queue and a searchable archive and folds evicted messages into a recursive summary that is always in context. The Mem0-style arm writes one compressed fact per turn with ADD, UPDATE and NOOP and evicts the least recently touched fact. Neither calls an LLM, so these are the systems' memory structures rather than their prompts. In the baselines' favour, their settings were grid-searched on LoCoMo itself at $S = 4096$; no MemGate arm was tuned on these answers.

Table 4.10 and Fig. 4.6 report the result. The learned scorer beats both rebuilds at every budget, by $+7.3$ to $+24.6$ points over Mem0-style and $+11.5$ to $+22.7$ over MemGPT-style, every gap significant. The untrained P1 heuristic beats MemGPT-style everywhere but loses to the tuned Mem0-style at $S = 4096$, the budget its settings were chosen on. Mem0-style stops improving at 52.6% from $S = 8192$, because UPDATE replaces a fact with its nearest neighbour and so discards distinct facts that merely look alike. MemGPT-style stays within about two points of plain RAG, because its summary holds gists whose answers were compressed away.

Table 4.10. Answer recall (%) on LoCoMo against rebuilt MemGPT-style and Mem0-style memory. Bold Δ: the interval by conversation excludes zero.

		MemGPT	Mem0			Δ vs. Mem0-style	
S	RAG	-style	-style	P1-S	P3	P1-S	P3
2048	18.3	18.7	26.4	27.8	**33.7**	+1.4	+7.3
4096	28.1	30.2	44.5	37.5	**52.0**	−7.0	+7.6
8192	44.3	46.4	52.6	57.8	**69.1**	+5.2	+16.5
16384	66.4	65.7	52.6	71.3	**77.2**	+18.7	+24.6



Context budget 2048. The learned scorer is fitted per held-out conversation (the other conversations plus the other corpus). The baselines' knobs were grid-searched on LoCoMo at S=4k, which favours them. LLM-free rebuilds of each system's memory management (memgate/external_baselines.py), not the products.

Figure 4.6. Answer recall against the store budget for MemGate and the rebuilt MemGPT-style and Mem0-style memories.

4.11 Chats of up to ten million tokens

LoCoMo conversations average about 19K tokens. BEAM [31] supplies user–assistant chats of 100K, 500K, 1M and 10M tokens, and MemBench [32] short agent trajectories. Each conversation was scored by a learned scorer fitted, under five folds by conversation, without it; BEAM 1M and 10M lie outside its training corpora altogether. The store budget for BEAM is $S = 26{,}000$, about 2.5% of a 1M-token chat.

Table 4.11 reports answer recall. The learned scorer beats RAG on all seven corpora with the interval excluding zero, by +36.5 points on BEAM 1M and +46.2 on BEAM 10M. On a million-token chat RAG keeps only the most recent turns, so almost nothing it is asked about survives. The one corpus on which the learned scorer is not the best is MemBench, whose short factual statements are what the P1 rules were written to find. BEAM answers are mostly paraphrases, so only a few questions per corpus are answer-recoverable, and the BEAM 10M row rests on 13 questions.

4.12 Ten times the training data

The scorer that the application first shipped was fitted on 58,616 turns. It was retrained on ten times that, adding BEAM 1M and 10M and MemBench, 594,000 turns in all. Each candidate was replayed on held-out conversations at three to five budgets per corpus, with the folds dealt in three different orders, 26,444 replays in all, under the rule that a shipped model must lose nowhere.

Table 4.11. Held-out answer recall (%) at each corpus's store budget, 5-fold by conversation. $P3_{12}$: the earlier recipe; $P3_{ship}$: the shipped ensemble. Bold: the interval excludes zero. ‡PersonaChat validation dialogues, never trained on.

Corpus	S	n	RAG	P1	$P3_{12}$	$P3_{12}-$RAG [95% CI]	$P3_{ship}$	$P3_{ship}-P3_{12}$ [95% CI]
LoCoMo	4096	715	28.1	37.5	50.3	+22.2 [+17.5, +27.6]	51.9	+1.5 [−0.2, +3.4]
REALTALK	4096	160	16.9	32.5	36.2	+19.4 [+7.5, +26.4]	36.2	0.0 [−3.3, +3.1]
BEAM 100K	26K	34	32.4	35.3	73.5	+41.2 [+14.3, +67.6]	73.5	0.0 [0.0, 0.0]
BEAM 500K	26K	55	32.7	32.7	63.6	+30.9 [+16.7, +44.8]	61.8	−1.8 [−10.0, +5.8]
BEAM 1M	26K	52	13.5	11.5	50.0	+36.5 [+21.8, +51.0]	50.0	0.0 [−7.3, +8.0]
BEAM 10M	26K	13	7.7	23.1	53.8	+46.2 [+10.0, +76.9]	38.5	−15.4 [−44.4, 0.0]
MemBench	2048	367	76.8	97.5	95.6	+18.8 [+14.2, +23.7]	97.5	+1.9 [+0.3, +3.8]
Everyday chat‡	1024	1762	28.8	30.4	61.4	—	69.0	+7.6 [+5.9, +9.5]

More data on its own did not help (Table 4.12, first column). The 10× model gained on MemBench but lost evidence recall on BEAM 1M and 500K in every shuffle. A linear model on 399 inputs was short of capacity, not of data, and the new turns were mostly negatives. Boosted trees, the obvious source of capacity, lost 8.5 points on LoCoMo and were rejected. Three changes put it right: weighting corpora within a family by their evidence turns, adding everyday chat from PersonaChat [56] as a fifth family (262,876 turns), and averaging the result in logit space with the earlier model, Eq. (3.5). The shipped model is trained on 856,876 turns, 14.6 times the original, and has no significant loss on any corpus at the paper's budgets; it gains +7.6 to +8.2 answer points on everyday chat.

Table 4.12. Retraining: change against the earlier recipe in three reshuffled fold orders, held out. "App path" adds the application's chunking, guard and multi-query retrieval. Bold: the interval excludes zero.

	10x (594K turns)	Shipped (857K turns)	
Corpus	Evidence	Answer	Evidence
LoCoMo	−0.4, −0.8, −0.2	+1.5, −0.3, +0.3	+0.1, −0.2, +0.2
REALTALK	+0.1, −1.6, 0.0	0.0, 0.0, +0.6	−0.3, −0.8, +0.2
BEAM 100K	−1.3, −0.8, −3.7	0.0, +2.9, −2.9	+1.8, +1.8, +0.6
BEAM 500K	−5.2, −4.4, −3.0	−1.8, +1.8, +1.8	+1.6, +2.0, +2.8
BEAM 1M	−5.1, −5.6, −5.3	0.0, −1.9, +1.9	+0.3, 0.0, −0.4
BEAM 10M	−2.3, −2.5, −0.5	−15.4, 0.0, −7.7	−1.2, −1.8, +1.2
MemBench	+5.5, +5.8, +6.2	+1.9, +1.9, +2.2	+5.6, +5.9, +6.4
Everyday chat‡	—	+7.6, +8.2, +8.0	+8.8, +9.1, +9.3
App path, BEAM 100K	+6.1, +4.2, +6.0	+11.8, +11.8, +8.8	+12.4, +11.5, +11.9
App path, LoCoMo	−0.5, −0.7, −0.2	+1.5, 0.0, +0.4	+0.3, +0.1, +0.5
App path, REALTALK	+0.7, +0.3, −0.3	+3.1, +1.9, +0.6	+0.5, −0.1, 0.0

4.13 Stress and scale through the application

The complete pipeline was also tested the way a user meets it. Whole BEAM chats were typed into the application one message at a time, exactly as its page sends them, at $S = 26{,}000$, and each answerable question was then asked as a message. Table 4.13 reports the result: on million-token chats, which MemGate holds at about 2.5%, the

answer reaches the reader for 51.9% of questions against 19.2% for RAG and 9.6% for a sliding window. The store never exceeded S, and a message took 0.49 s on average on a shared 4-core machine.

Table 4.13. Whole BEAM chats typed into the application one message at a time, $S = 26{,}000$; answer recall (%).

Chats	N	M tok.	n	P0	RAG	MemGate	Δ vs. RAG [95% CI]	McNemar p
BEAM 1M	33	1.07	52	9.6	19.2	**51.9**	+32.7 [+19.2, +46.3]	9.1×10^{-4}
BEAM 500K	33	0.52	55	12.7	30.9	**65.5**	+34.5 [+19.6, +49.1]	3.1×10^{-4}

Much of the gap comes down to where the evidence sits (Table 4.14). RAG forgets oldest first, so for evidence in the first third of a conversation it keeps the answer 8–9% of the time; the untrained heuristic keeps 20–33%, and the learned scorer more. For evidence in the last third RAG is competitive, since recency is then the right rule.

Table 4.14. Answer recall (%) by the position of a question's evidence, $S = 4096$. *In-sample on both corpora.

Corpus	Evidence in	n	RAG	P1	P3*
LoCoMo	first	262	9.2	33.2	51.9
	middle	225	14.2	38.2	53.8
	last	228	63.6	41.7	54.4
REALTALK	first	86	8.1	19.8	33.7
	middle	45	11.1	40.0	55.6
	last	29	51.7	58.6	65.5

People rarely ask one clean question per message. When each LoCoMo question was buried in 150–450 words of other chat, using the whole message as the query dropped answer recall from 53.1% to 30.3%; making each question its own query with interleaved rankings recovers 49.8%.

The same tests found six input-handling bugs, each now pinned by a test: half of an emoji in one message made every later save of that chat fail; a 300 KB Word file that unzips to 300 MB used 3.8 GB of memory; deeply nested JSON dropped the connection; request bodies had no size limit; a message of only invisible characters was accepted; and a locked PDF gave an unclear error. A load test found one more: under bursts, the server's accept queue of five reset 314 of 2,000 requests from 200 simultaneous visitors. With a queue of 256 the same test completes with no errors in 25 s, and a single chat of 5,000 messages stays at 4,093 of its 4,096 tokens, at 0.12 s per message.

4.14 Standards followed

The following standards and practices were adhered to in building and testing the system and in validating its results.

- **IEEE Std 754 (floating-point arithmetic).** All embeddings and model weights are IEEE 754 single-precision (`float32`) values; the 1,536-byte cost of a vector in Eq. (3.8) is 384×4 bytes of this format.
- **ISO/IEC/IEEE 29119 (software testing) and IEEE Std 1012 (verification and validation).** Testing follows their structure: unit tests for each module, integration tests through the application's API, and a benchmark-level validation test that replays LoCoMo and compares it with the published numbers; regression tests pin every fixed defect.
- **ISO/IEC/IEEE 29148 (requirements engineering).** The scope, in-scope and out-of-scope items and the measurable objectives of Chapter 1 were written as verifiable requirements, each checked by an experiment or a test.
- **Unicode and UTF-8 (RFC 3629).** All text is stored as UTF-8 and NFC-normalised; control characters, zero-width marks and bidirectional overrides are removed on input.
- **RFC 9110 (HTTP semantics) and RFC 8259 (JSON).** The application's server uses standard HTTP methods and status codes and exchanges JSON.
- **Statistical practice.** Paired bootstrap confidence intervals clustered by conversation [37, 38] and exact McNemar tests [35, 36]; a result is claimed only when its interval excludes zero.
- **Reproducibility.** Every model is a pinned local copy; every table is generated from committed result files; and the leakage invariant (no decision may read an evaluation label) is enforced by tests [39, 40].

Summary

Once the store is bounded, deciding what to forget is worth $+9.37$ answer points over FIFO, while compressing instead of dropping costs 7.13; no rate–distortion crossover appears up to $182\times$; and counting the index in bytes reverses the ranking below about 40 KiB. An LLM judge loses to a heuristic, but a learned scorer beats the heuristic by $+14.55$ points on held-out conversations, transfers from human to generated dialogue, beats MemGPT-style and Mem0-style memory at every budget, and keeps the answer for 51.9% of questions on million-token chats against 19.2% for retrieval. Two results are negative: the heuristic fails on LongMemEval, and ten times the data alone did not improve the scorer.

Chapter 5

Advantages, Limitations and Applications

This chapter steps back from individual results to assess MemGate as a system: what it does better than the alternatives, where it falls short or has not yet been tested, and where it can be applied.

5.1 Advantages

1. **Bounded cost on every turn.** The store never exceeds its budget S and the prompt never exceeds B, however long the conversation grows. In the application a 5,000-message chat stayed at 4,093 of its 4,096 tokens, and million-token chats were held at about 2.5% of their length.
2. **Better recall at the same budget.** At identical storage, fidelity and retrieval, scored forgetting beats oldest-first forgetting by $+9.37$ answer points, and the learned scorer adds a further $+14.55$ over the heuristic on held-out conversations. It beats rebuilt MemGPT-style and Mem0-style memory at every budget tested.
3. **Works with any model.** The base LLM is a black box. MemGate needs no training of the model, so it can sit in front of a local model, a hosted API, or a newer model later, unchanged.
4. **Cheap to run.** Scoring a message is a 399-weight dot product plus one sentence embedding; there is no LLM call on the write path. The whole layer runs on a laptop CPU, and the shipped scorer is a small JSON file.
5. **Transparent decisions.** Every keep, compress and forget decision has a score and an explanation that a user can inspect, which an LLM-driven memory cannot easily offer.
6. **Measured, not asserted.** Each design choice was tested against a control that differs in one factor, with clustered confidence intervals, and the results that did not hold are reported as such.
7. **Private and offline.** With the local reply model, the memory and the chats stay on the user's machine and nothing needs the internet after setup.
8. **Robust input handling.** Long pastes, documents, other languages, emoji and code are cleaned, chunked and guarded before they reach the scorer, and the

server held 200 simultaneous visitors without an error after the load-test fix.

5.2 Limitations

1. **Context recall is not accuracy.** Most results measure whether the answer reached the context. End-task accuracy was measured through one small reader model, and that run, with the LLM-judge, abstractive and LongMemEval runs, was made before the scorer fix and has not been repeated.
2. **The heuristic does not transfer to every corpus.** On LongMemEval the hand-written scorer's separation between evidence and other turns reverses sign and selection loses to FIFO. The learned scorer has not yet been tested there.
3. **Limited statistical power on human conversation.** REALTALK has 160 answer-recoverable questions in ten conversations, so the learned scorer's gains there are significant only once the store holds about 40% of a conversation.
4. **Proxies in the scale tests.** BEAM answers are mostly paraphrases, so few questions per corpus are answer-recoverable (13 for BEAM 10M), and the everyday-chat test uses persona lines as queries, which share wording with the evidence.
5. **A linear scorer.** Logistic regression was kept because nothing beat it reliably, but it is short of capacity: ten times more data did not help on its own.
6. **Text only.** Images, audio and tool outputs that are not text are out of scope.
7. **One slow path.** Reopening a saved million-token chat replays it through memory and takes about 70 s.
8. **Dependence on the reply model.** When the free hosted model is busy the application falls back to a small local model, whose answers are weaker even when the right notes are supplied.

5.3 Applications

1. **Personal and customer-support assistants** that must remember a user's details, preferences and past requests across weeks without sending the whole history on every call.
2. **Coding and research agents** whose working history (files read, decisions taken, errors seen) grows far beyond the context window.
3. **On-device assistants.** The byte-level results show which policy to use when the memory budget is a few tens of kilobytes, as on a phone or an embedded device.

4. **Education and tutoring**, where a tutor should recall what a student has already learnt and where they struggled.
5. **Healthcare and legal intake assistants**, where specific names, dates and amounts must be kept while small talk can be forgotten (subject to the privacy rules of those domains).
6. **Document question answering** over long uploaded files, using the chunk, score and recall path of the application.
7. **Evaluating other memory systems.** The harness itself can be used to measure any memory policy against controls under a bounded store.

Summary

MemGate keeps cost bounded, improves recall at the same budget, works with any model, runs cheaply and explains its decisions. Its main limitations are that some results are measured as context recall or before the scorer fix, that transfer to every corpus is not established, and that the scorer is linear. It applies wherever a long-running assistant must remember specific facts under a cost or memory limit.

Chapter 6

Conclusion and Future Scope

This chapter summarises the work carried out, the conclusions drawn from the results of Chapter 4, and the directions in which the work can be taken further.

6.1 Conclusion

This project set out to measure the memory decision of an LLM agent: what to keep, what to compress, what to forget and what to recall. It built MemGate, a memory layer in front of a frozen model, as a harness in which each of these decisions can be swapped and compared against a control that differs in exactly one factor, under a store held to a fixed budget.

Under the accounting this field normally uses, tiered compression loses to storing everything and retrieving. That result is an artefact of leaving the store unbounded. Once the store is bounded, deciding which items to forget is worth $+9.37$ answer-recall points at identical storage, fidelity and retrieval, while compressing an evicted item instead of dropping it costs 7.13. The point at which rate–distortion theory says compression should start to pay is not reached by $182\times$. Counting the embedding index in the budget reverses the ranking below about 40 KiB.

The obvious route to a better scorer, an LLM judge, loses to a regular expression, because salience is not a property of a turn read on its own. A logistic scorer over a sentence embedding, trained on generated and human conversations together, beats the heuristic by $+14.55$ points on held-out LoCoMo conversations and by a significant margin at every store budget from 2048 to 16384 tokens. It transfers from human dialogue to generated dialogue, but not the other way round. Through a fixed reader, $+6.0$ points of the recall advantage survive as correct answers. The same scorer beats MemGPT-style and Mem0-style memory at every budget tried, beats retrieval on held-out chats of up to ten million tokens, and, when million-token chats are typed into the application, keeps the answer 51.9% of the time against 19.2%. Ten times the training data did not, on its own, produce a better linear scorer; weighting corpora by their labels, adding everyday chat and averaging with the earlier model did.

The practical advice from this work is narrow but well supported: keep a scored subset at full fidelity rather than compressing everything, and count the memory budget in bytes with the index included. The learned scorer ships in a working chat application

that makes every one of these decisions visible to its user, and the findings have been written up as a research paper.

6.2 Future scope

1. **Re-run the pre-fix experiments.** The LLM-judge, end-task, abstractive and LongMemEval runs were made before the scorer fix and should be repeated with the fixed heuristic and the shipped learned scorer.
2. **LongMemEval with the learned scorer.** The heuristic inverts on LongMemEval; whether the learned scorer does too is the most important open test of transfer.
3. **More human-written evidence.** The learned scorer's gains on human conversation are significant only at larger budgets. More annotated human conversations, rather than a larger model, are what would settle this.
4. **Personalised online learning.** A scorer learned online from retrieval feedback is already level with the heuristic with no labels. Starting it from the pooled scorer and letting it adapt to each user is the step from a scorer fitted to benchmarks to one fitted to the person using the application.
5. **More capacity without losing speed.** A small non-linear scorer that keeps the write path cheap, tested under the same lose-nowhere rule.
6. **Multimodal memory.** Extending the tiers and the scorer to images, audio and tool outputs.
7. **Deployment.** Faster reopening of very long saved chats, multi-user hosting, and integration as a memory module for agent frameworks.

Summary

Selection beats compression under a bounded store, a learned scorer beats both a heuristic and an LLM judge, and the result holds through a reader, against existing memory designs and at million-token scale. The open questions are transfer to every corpus and adaptation to each user.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[2] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[3] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[4] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou *et al.*, "The rise and potential of large language model based agents: A survey," *arXiv preprint arXiv:2309.07864*, 2023.

[5] T. B. Brown, B. Mann, N. Ryder *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[6] B. Peng, J. Quesnelle, H. Fan, and E. Shippole, "YaRN: Efficient context window extension of large language models," in *International Conference on Learning Representations (ICLR)*, 2024.

[7] Chroma Research, "Context rot: How increasing input tokens impacts LLM performance," Chroma, Tech. Rep., 2025.

[8] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg, "RULER: What's the real context size of your long-context language models?" in *Conference on Language Modeling (COLM)*, 2024.

[9] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[10] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023.

[11] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, "MemoryBank: Enhancing large language models with long-term memory," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19 724–19 731.

[12] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. ACM Symp. User Interface Software and Technology (UIST)*, 2023.

[13] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," *arXiv preprint arXiv:2504.19413*, 2025.

[14] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[15] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, "Cognitive architectures for language agents," *Transactions on Machine Learning Research*, 2024.

[16] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking chat assistants on long-term interactive memory," *arXiv preprint arXiv:2410.10813*, 2024.

[17] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, and J.-R. Wen, "A survey on the memory mechanism of large language model based agents," *arXiv preprint arXiv:2404.13501*, 2024.

[18] C. E. Shannon, "Coding theorems for a discrete source with a fidelity criterion," in *IRE National Convention Record*, vol. 4, 1959, pp. 142–163.

[19] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.

[20] R. Colaco and A. Lahjouji, "A rate–distortion framework for memory in language agents," *arXiv preprint arXiv:2607.08032*, 2026.

[21] Y. Bai, X. Lv, J. Zhang *et al.*, "LongBench: A bilingual, multitask benchmark for long context understanding," in *Proc. Annu. Meeting Association for Computational Linguistics (ACL)*, 2024, pp. 3119–3137.

[22] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[23] S. Borgeaud, A. Mensch, J. Hoffmann *et al.*, "Improving language models by retrieving from trillions of tokens," in *Proc. Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 162, 2022, pp. 2206–2240.

[24] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[25] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 5776–5788.

[26] R. C. Atkinson and R. M. Shiffrin, "Human memory: A proposed system and its control processes," *Psychology of Learning and Motivation*, vol. 2, pp. 89–195, 1968.

[27] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.

[28] P. J. Denning, "The working set model for program behavior," *Communications of the ACM*, vol. 11, no. 5, pp. 323–333, 1968.

[29] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of LLM agents," in *Proc. Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2024.

[30] D.-H. Lee, A. Maharana, J. Pujara, X. Ren, and F. Barbieri, "REALTALK: A 21-day real-world dataset for long-term conversation," *arXiv preprint arXiv:2502.13270*, 2025.

[31] M. Tavakoli *et al.*, "Beyond a million tokens: Benchmarking and enhancing long-term memory in LLMs," in *International Conference on Learning Representations (ICLR)*, 2026, arXiv:2510.27246.

[32] H. Tan *et al.*, "MemBench: Towards more comprehensive evaluation on the memory of LLM-based agents," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, arXiv:2506.21605.

[33] L. Zheng, W.-L. Chiang, Y. Sheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023.

[34] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2016, pp. 2383–2392.

[35] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[36] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.

[37] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[38] A. C. Cameron, J. B. Gelbach, and D. L. Miller, "Bootstrap-based improvements for inference with clustered errors," *Review of Economics and Statistics*, vol. 90, no. 3, pp. 414–427, 2008.

[39] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

[40] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.

[41] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-MEM: Agentic memory for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[42] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 13 358–13 376.

[43] J. Mu, X. L. Li, and N. Goodman, "Learning to compress prompts with gist tokens," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[44] A. Chevalier, A. Wettig, A. Ajith, and D. Chen, "Adapting language models to compress contexts," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[45] T. Ge, J. Hu, L. Wang, X. Wang, S.-Q. Chen, and F. Wei, "In-context autoencoder for context compression in a large language model," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[46] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[47] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, "H2O: Heavy-hitter oracle for efficient generative inference of large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[48] J. W. Rae, A. Potapenko, S. M. Jayakumar, C. Hillier, and T. P. Lillicrap, "Compressive transformers for long-range sequence modelling," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020.

[49] Y. Wu, M. N. Rabe, D. Hutchins, and C. Szegedy, "Memorizing transformers," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[50] W. Wang, L. Dong, H. Cheng, X. Liu, X. Yan, J. Gao, and F. Wei, "Augmenting language models with long-term memory," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[51] F. Pedregosa, G. Varoquaux, A. Gramfort *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[52] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2025.

[53] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[54] M. Mintz, S. Bills, R. Snow, and D. Jurafsky, "Distant supervision for relation extraction without labeled data," in *Proc. Joint Conf. 47th Annu. Meeting ACL and 4th IJCNLP*, 2009, pp. 1003–1011.

[55] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*. MIT Press, 1999, pp. 61–74.

[56] S. Zhang, E. Dinan, J. Urbanek, A. Szlam, D. Kiela, and J. Weston, "Personalizing dialogue agents: I have a dog, do you have pets too?" in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018, pp. 2204–2213.

Appendix A

Soft Code Flowcharts

This appendix gives the flowcharts of the two paths a message takes through the MemGate application: storing it (Fig. A.1) and answering from memory (Fig. A.2). The routing and budget loops inside the library are shown in Fig. 3.4, the scorer's guard in Fig. 3.3, and the training of the shipped scorer in Fig. 3.2.

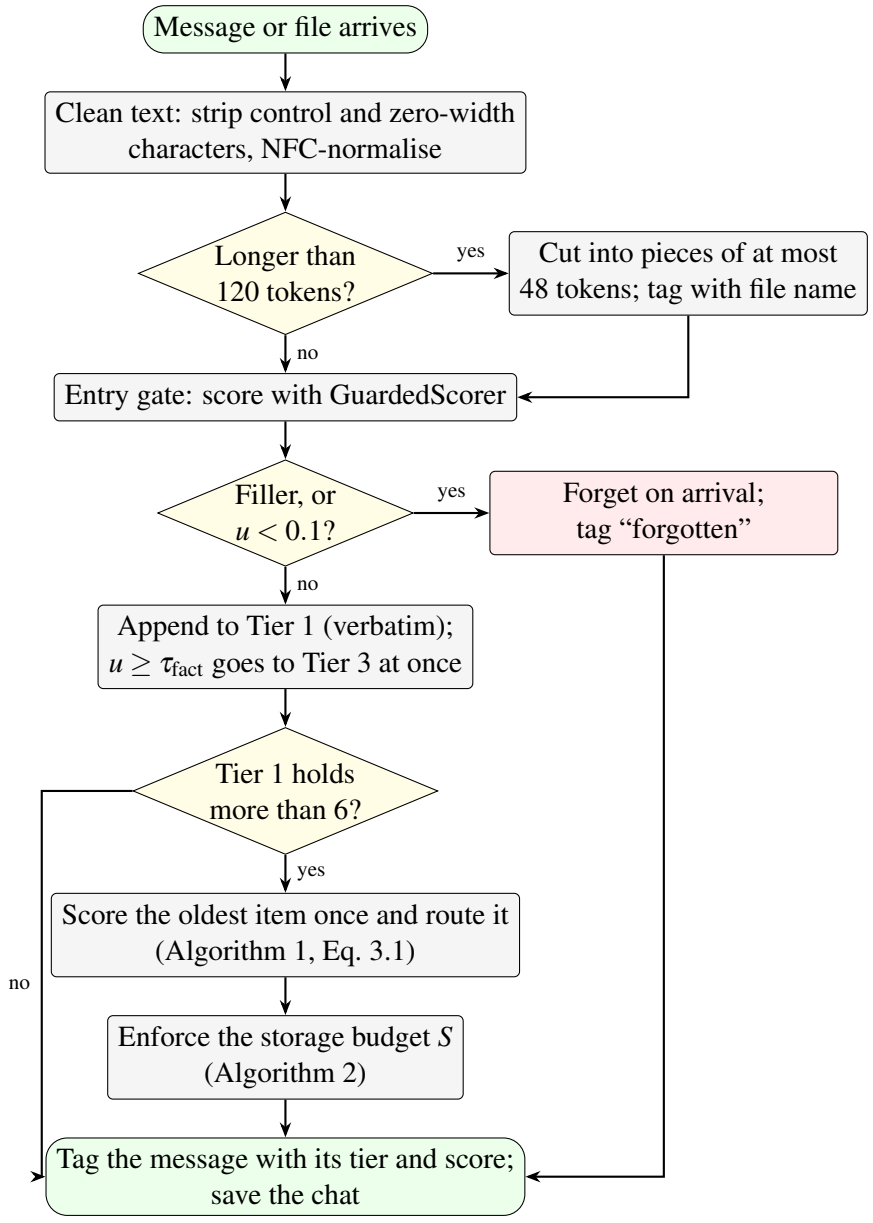


Figure A.1. Flowchart of the write path in the application: how one message is cleaned, gated, stored and kept within budget.

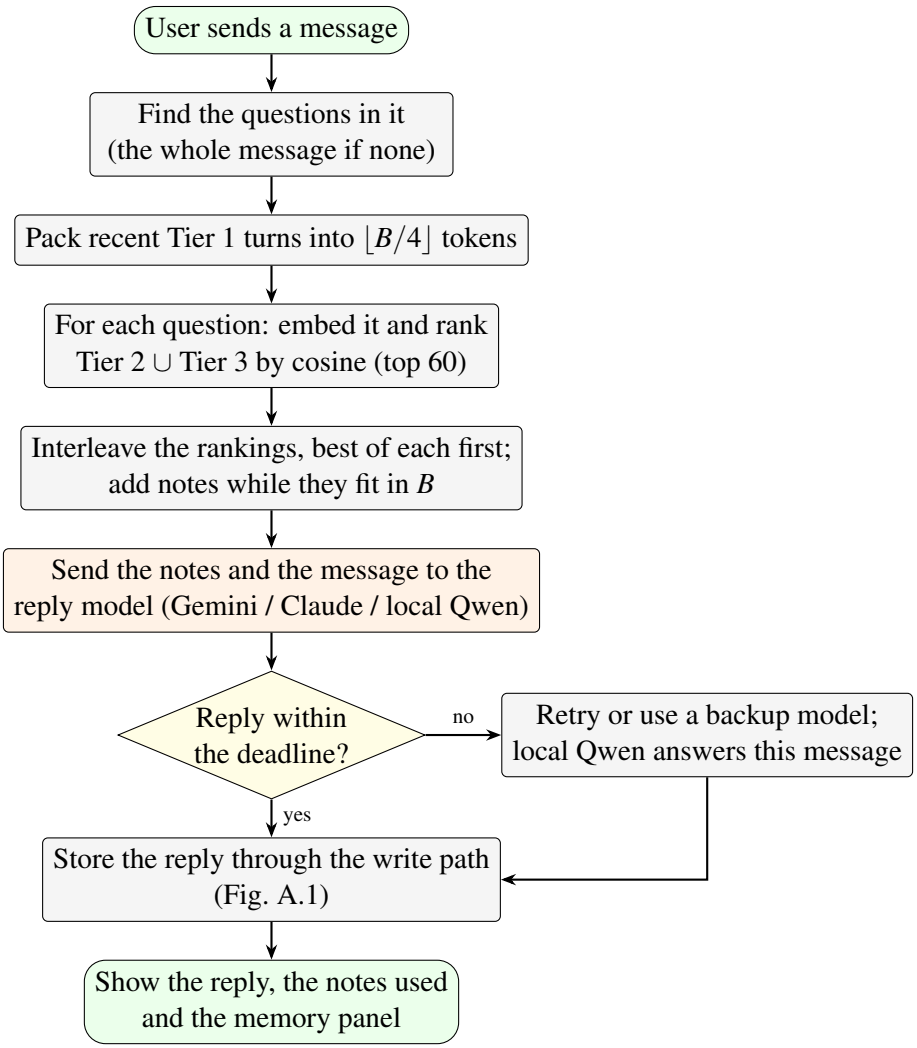


Figure A.2. Flowchart of the read path in the application: how a reply is assembled from memory.

Appendix B

Data Sheets

MemGate uses no hardware components, so its data sheets describe the datasets and models it depends on (Tables B.1 and B.2). All datasets are public and were used under their published licences; no personal data was collected by the project.

Table B.1. Datasets used.

Dataset	Size	Use in this project
LoCoMo [29]	10 conversations, 5,882 turns, 1,527 questions (715 answer-recoverable)	Main benchmark; LLM-generated multi-session dialogue with marked evidence
REALTALK [30]	10 conversations, 8,944 turns, 660 questions (160 answer-recoverable)	Human conversation over 21 days; transfer test and training
BEAM 100K / 500K [31]	20 and 35 chats	Long user–assistant chats; training and held-out tests
BEAM 1M	35 chats, 74,630 turns, 622 questions with evidence	Held-out million-token test; app-path test
BEAM 10M	10 chats, 208,696 turns, 128 pinned questions	Held-out ten-million-token test
MemBench [32]	4,330 trajectories, 252,058 turns	Short factual agent memories; training and test
PersonaChat [56]	894 long chats (262,876 turns) train; 50 chats test	Everyday chat with weak labels from persona lines
LongMemEval [16]	94 histories of 38–62 sessions	Second-benchmark test of the headline

Table B.2. Models and tokeniser used.

Model	Size	Role
all-MiniLM-L6-v2 [24, 25]	22.7M parameters, 384-d output, about 90 MB	Embeddings for retrieval, dedupe, supersession and the learned scorer
Learned scorer P3 (this work)	399 weights + bias, plain JSON	Utility of each message
Qwen2.5-0.5B-Instruct [52]	0.5B parameters	LLM judge of salience (P2)
Qwen2.5-1.5B-Instruct [52]	1.5B parameters (4-bit on Apple silicon)	End-task reader; abstractive compressor; offline reply model
Google Gemini (flash) [53]	hosted	Default reply model in the application (free key)
`cl100k_base` (tiktoken)	100K-token vocabulary	Exact token counts for every budget

Appendix C

List of Components

MemGate has no hardware components. Its software components are listed in Tables C.1 and C.2. The source code is in the project repository (branch `endtask-full-run`).

Table C.1. Software components: main files of the repository.

File or folder	Purpose
`memgate/memgate/`	The library (Table 3.3)
`memgate/app.py`, `web/chat.html`	The chat application: server and page
`run_locomo.py`, `run_storage_sweep.py`	LoCoMo evaluation and the storage-budget sweep
`run_ablations.py`, `run_significance.py`	Component ablations; bootstrap and McNemar tests
`run_scaling.py`, `run_cost_model.py`	Compression-ratio scaling; byte accounting
`train_scorer.py`, `train_transfer.py`, `retrain_10x.py`, `export_scorer.py`	Training, transfer tests, retraining and export of the learned scorer
`run_judge_eval.py`, `run_endtask.py`	LLM-judge comparison; end-task accuracy
`run_longmemeval.py`, `run_abstractive.py`, `run_online.py`	LongMemEval; abstractive compression; online learning
`run_external_baselines.py`	MemGPT-style and Mem0-style comparison
`scale_app.py`, `stress_inputs.py`, `stress_questions.py`	Application scale, load and stress tests
`test_*.py`, `run_tests.py`	Test suites and their runner
`setup.sh/.ps1`, `start.sh/.bat`	One-command setup and start
`deploy/`	Docker image for an online Hugging Face Space
`paper/`	Research paper sources, generated tables and diagrams

Table C.2. Software libraries.

Library	Use
NumPy	Vectors and numerics
tiktoken	Token counting
sentence-transformers / ONNX Runtime	MiniLM embeddings
huggingface_hub	One-time model downloads
PyTorch, Transformers, mlx-lm	Local Qwen models (judge, reader, offline replies)
scikit-learn	Training the learned scorer
Matplotlib	Figures
pypdf	Reading uploaded PDF files
anthropic (optional)	Claude replies
Python standard library	HTTP server, JSON, Word/PowerPoint reading, statistics

Appendix D

Plagiarism and AI-Content Check Report

This appendix holds the similarity and AI-writing check of this report, run through Turnitin. The figures below are filled in by hand from the Turnitin report, and the report's summary page is inserted after this page.

Table D.1. Turnitin check summary

Item	**Value**
Tool	Turnitin
Date of check	
Submission ID	
Overall similarity index (%)	
AI-writing detection (%)	
Excluded from the check	

Dr. Praveen Modi
(Internal Guide)

Turnitin similarity and AI-writing report

(insert the printed report here)